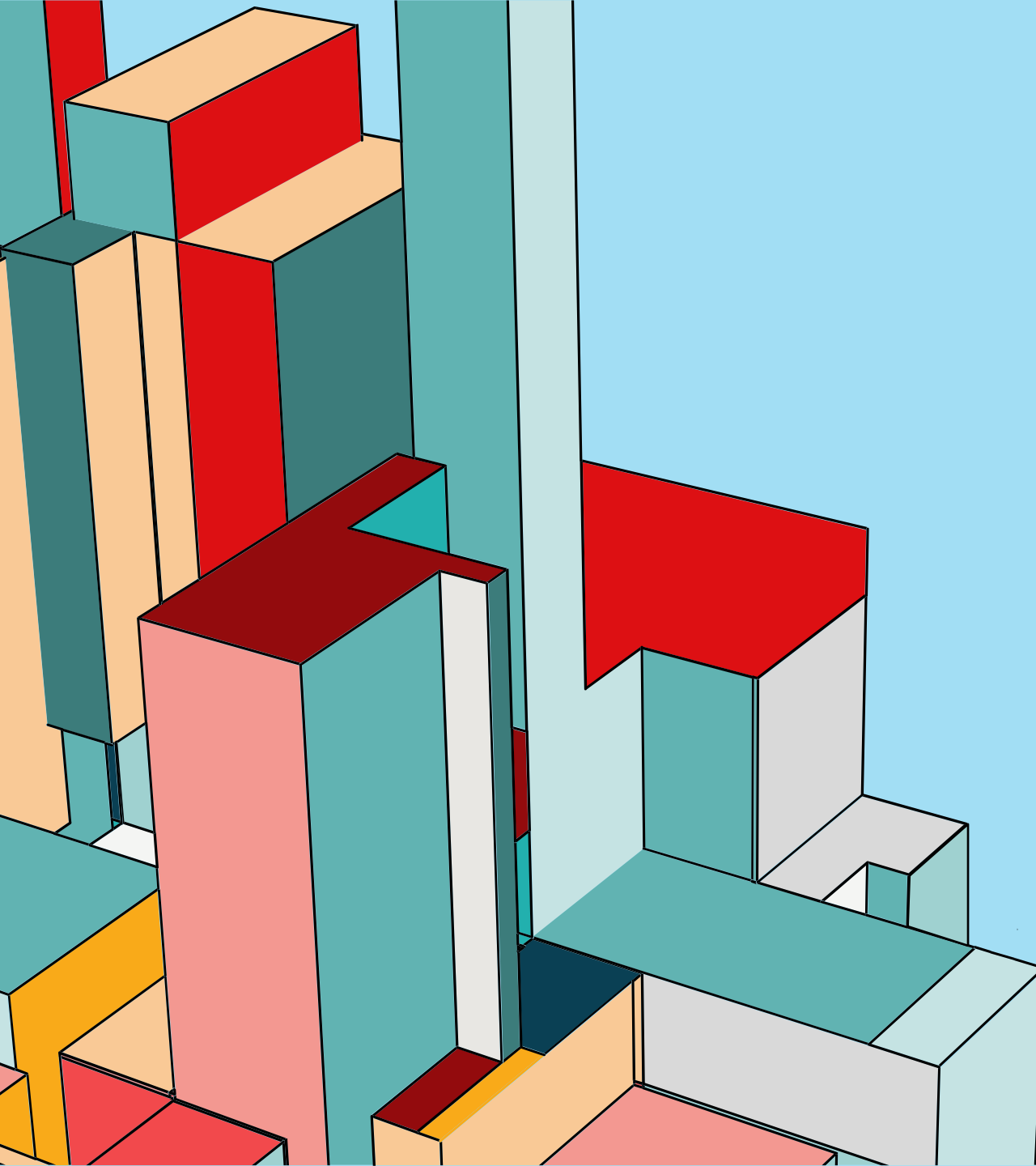




LECTURE 20 **FLOW NETWORKS**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

PROBLEM OF THE DAY

Consider a graph in which each edge has a certain capacity.

You might ask: What is the maximum number of passengers that could be moved from Washington Square to the Barclay's Center in, say, an hour?

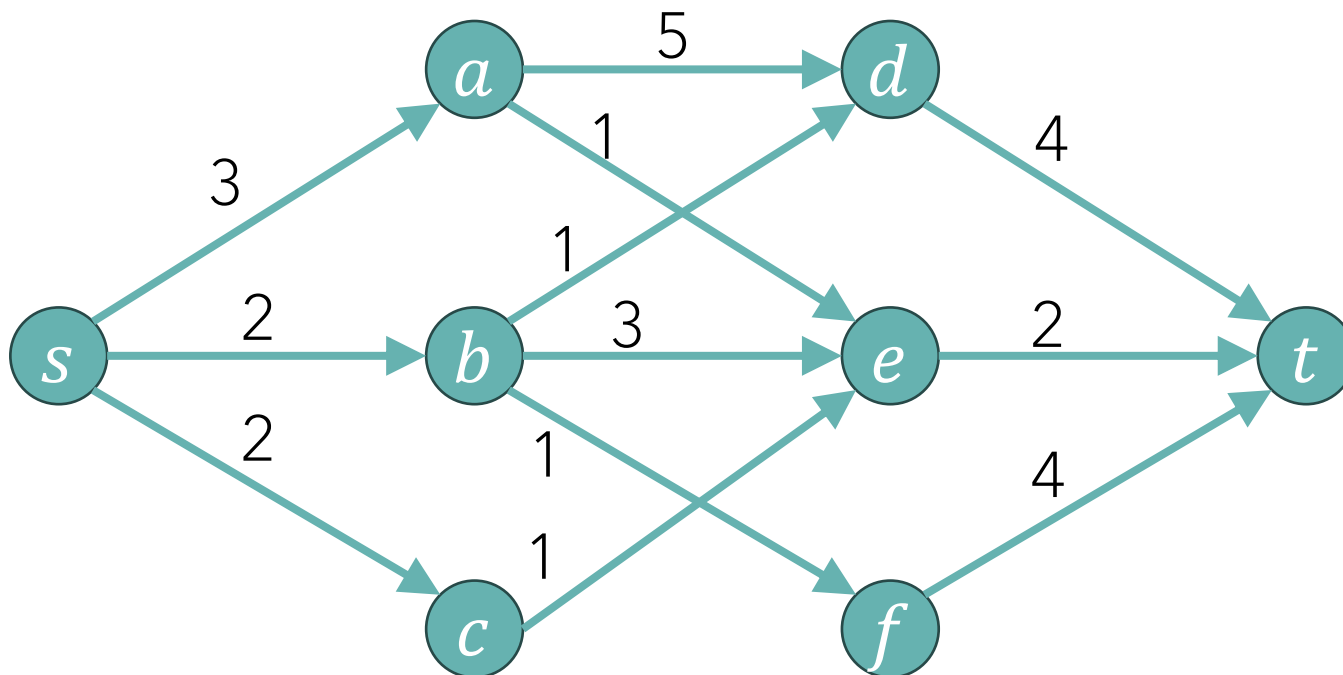


Image credit: MTA

This is known as the *Maximum Network Flow* problem – *MaxFlow* for short.

SUBMITTY POLL #1

Consider the graph below in which the weight on each edge represents the capacity of that edge. How many units of stuff can this network deliver from s to t ? Answer now in the Submittity poll.



FORMALIZING THE PROBLEM

Let G be a connected, weighted, directed graph in which the weights $c(e)$ represent capacities, with all $c(e) > 0$. Further:

- There is one vertex s with no incoming edges: the *source*
- There is one vertex t with no outgoing edges: the *sink*
- Every other vertex has at least one incoming edge and one outgoing edge; these are called *internal nodes*.

We call a graph meeting these conditions a *flow network*.

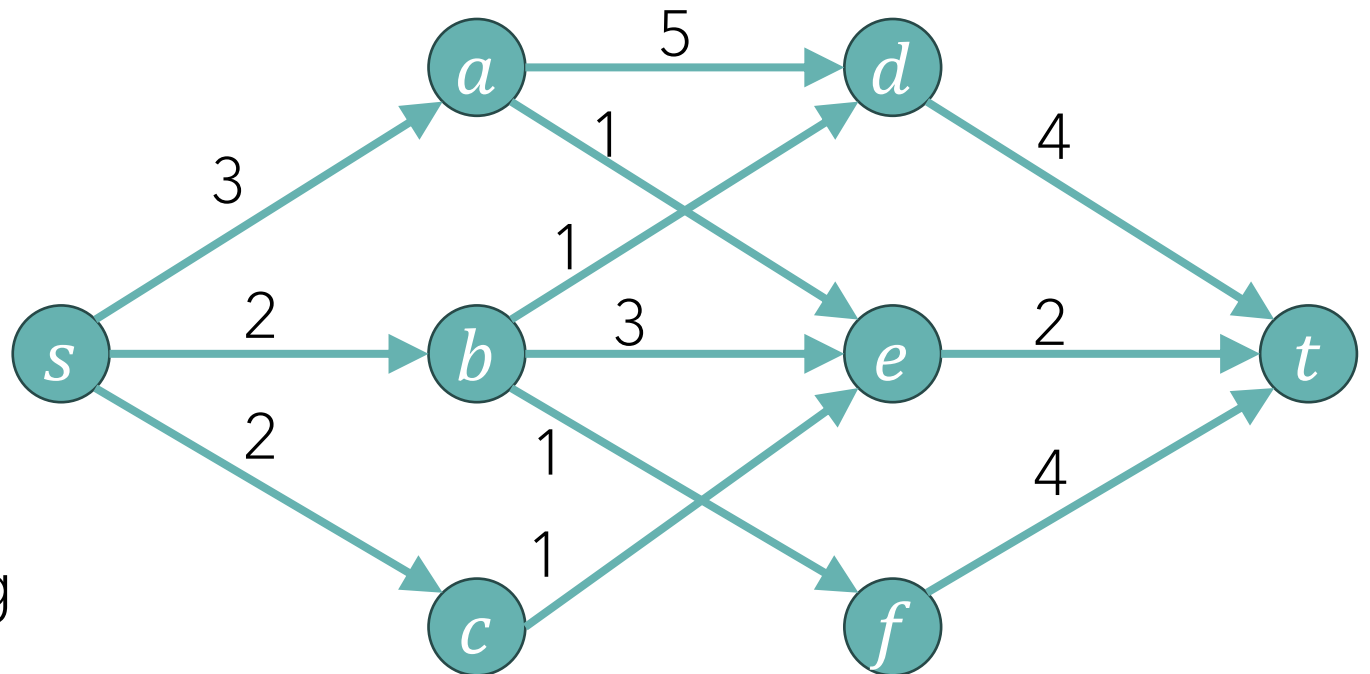
Next, in order to talk about the maximum flow, we need to define what a *flow* is in the first place.

FORMALIZING THE PROBLEM

Intuitively, the *flow* is the amount of “stuff” moving through each edge – that is, f is a function that maps edges to amounts.

What restrictions should we place on f ?

- *Capacity*: $f(e) \leq c(e)$
- *Direction*: $f(e) \geq 0$
- *Conservation*: For any internal node, the total flow on its incoming edges should equal the total flow on its outgoing edges.



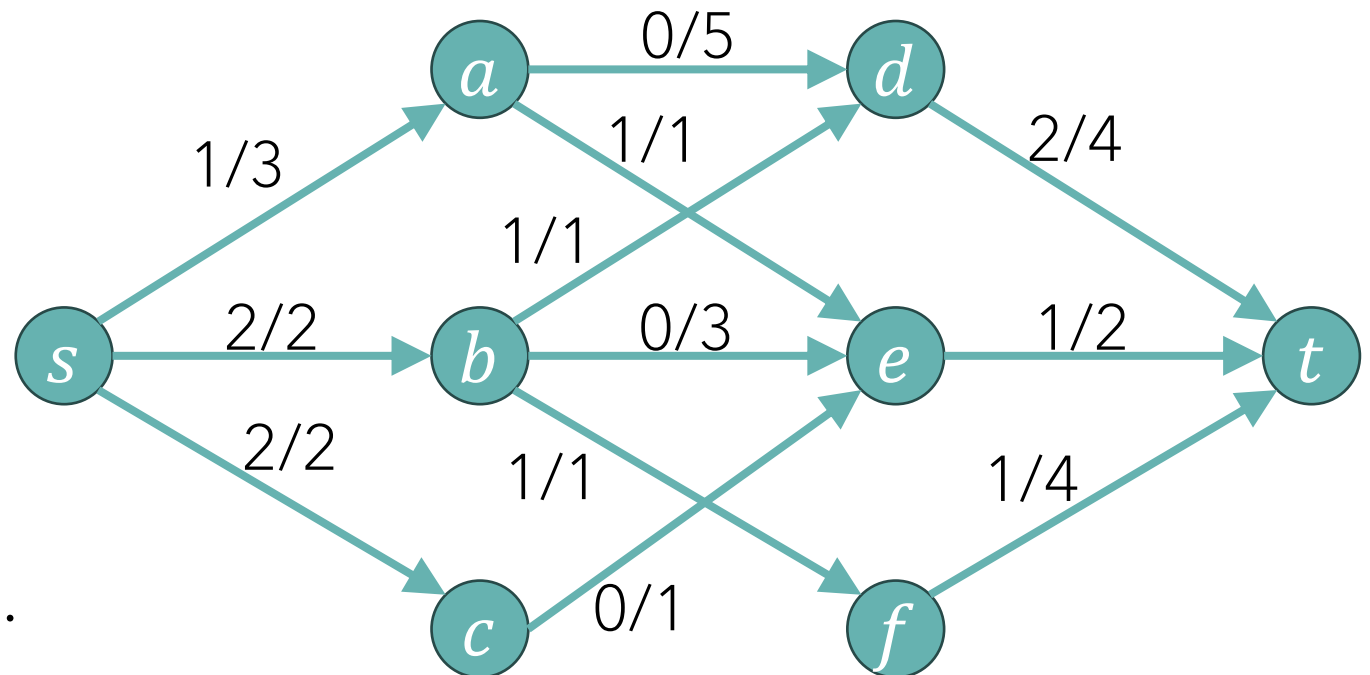
AN INVALID FLOW

Note that node c has 2 flow coming into it, but none coming out.

Conversely, node d only has 1 flow coming in, but 2 coming out.

For now, we will stick to the most basic version of the problem, in which s is the only source of flow and t is the only sink.

This means that every unit of flow that leaves s must eventually arrive at t .



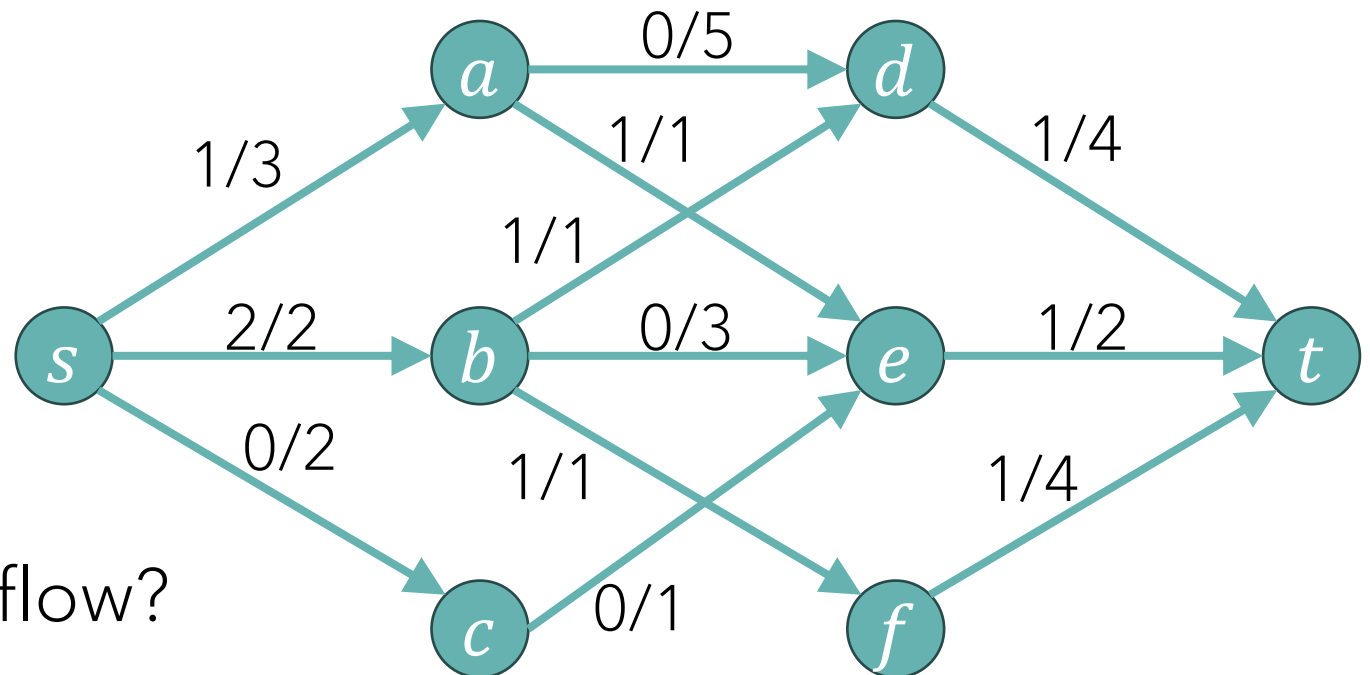
THE MAX-FLOW PROBLEM

Shown here is one possible valid flow. The *value* or *volume* of a flow is the sum of all flow leaving s (equivalently, arriving at t).

The *Max-Flow Problem* asks: Given a flow network G , find a valid flow f with largest value.

For the time being, we will impose the further restriction that all of the capacities are integers.

What algorithm might we use to find the maximum flow?

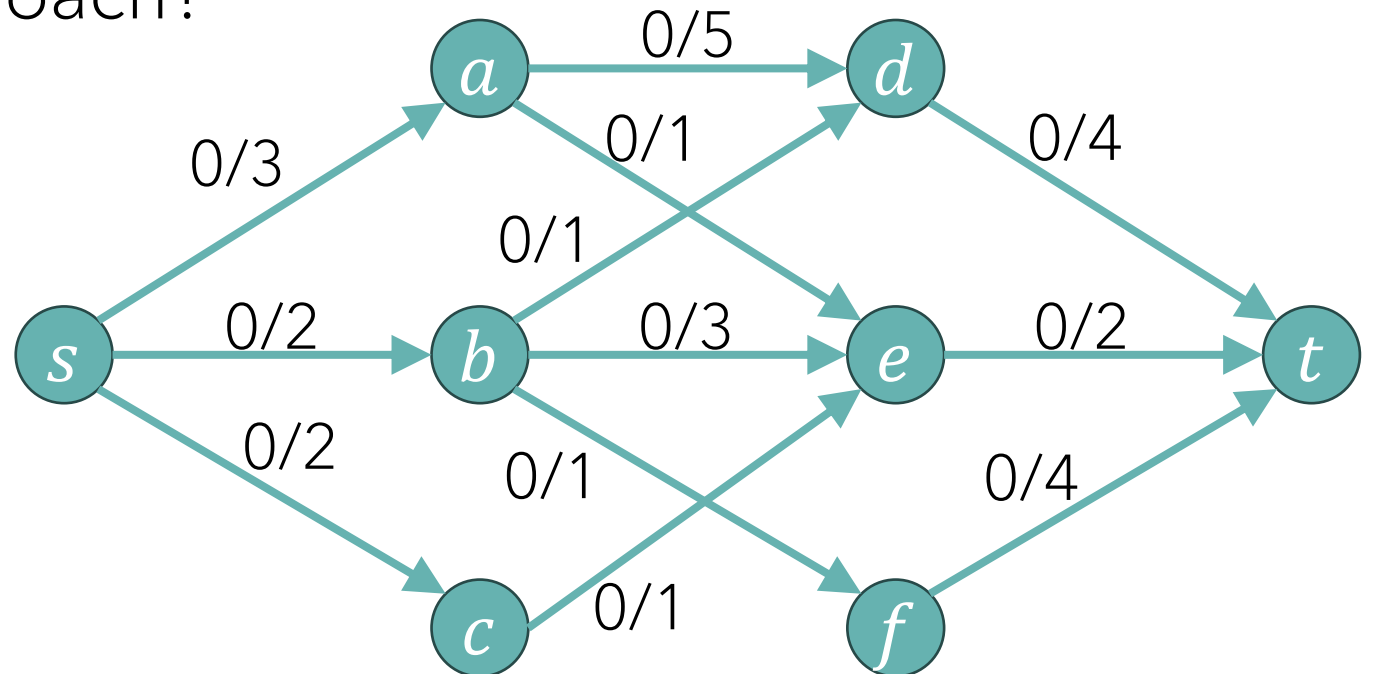


TRYING THE BASICS

We can observe that brute force here just isn't going to be a thing – that would involve something like trying every possible value for every edge, and then checking validity. WAY too huge.

How about a greedy approach?

Note that you can't just greedily select the edge with largest capacity; this is fairly likely to violate conservation in a way that isn't fixable.



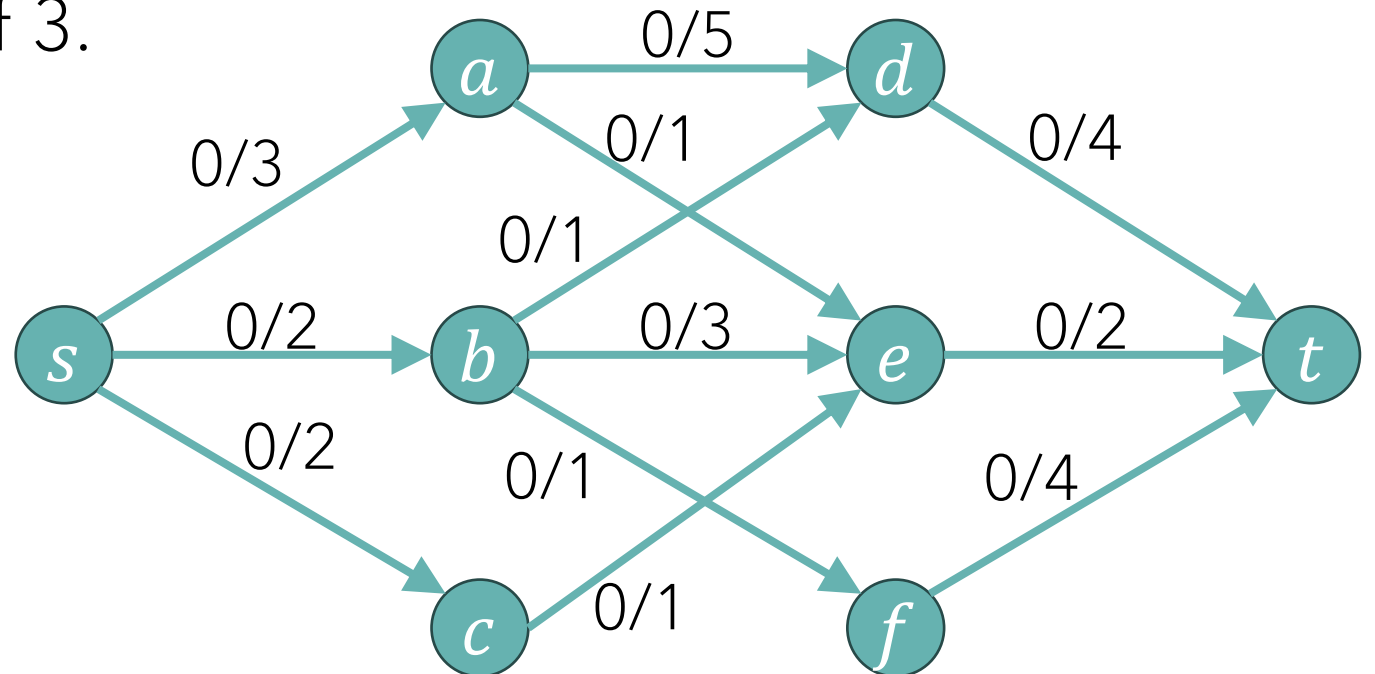
But we can pick a path...

TRYING THE BASICS

We know every unit of flow must originate at s and end up at t .

What if we greedily select the highest-capacity path from s to t ?

Our first selection would be
 $s \rightarrow a \rightarrow d \rightarrow t$ with flow of 3.



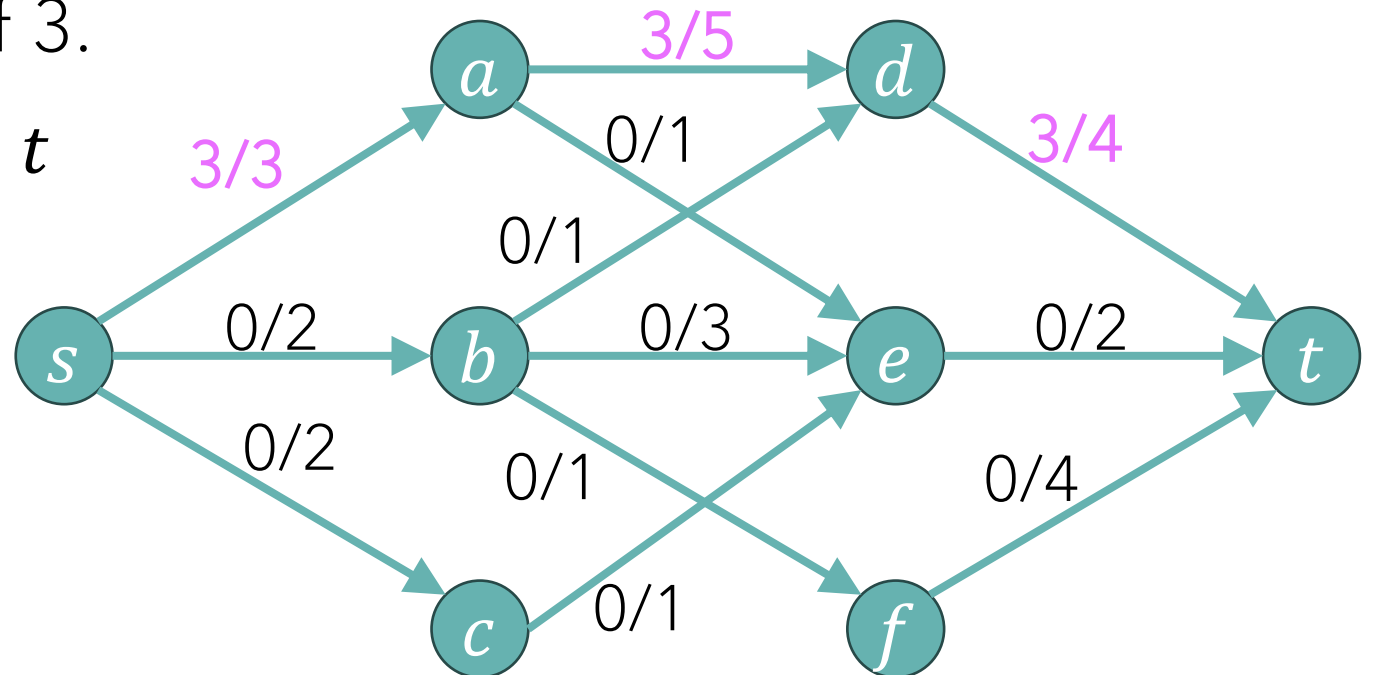
TRYING THE BASICS

We know every unit of flow must originate at s and end up at t .

What if we greedily select the highest-capacity path from s to t ?

Our first selection would be $s \rightarrow a \rightarrow d \rightarrow t$ with flow of 3.

Next we take $s \rightarrow b \rightarrow e \rightarrow t$ with flow of 2.



TRYING THE BASICS

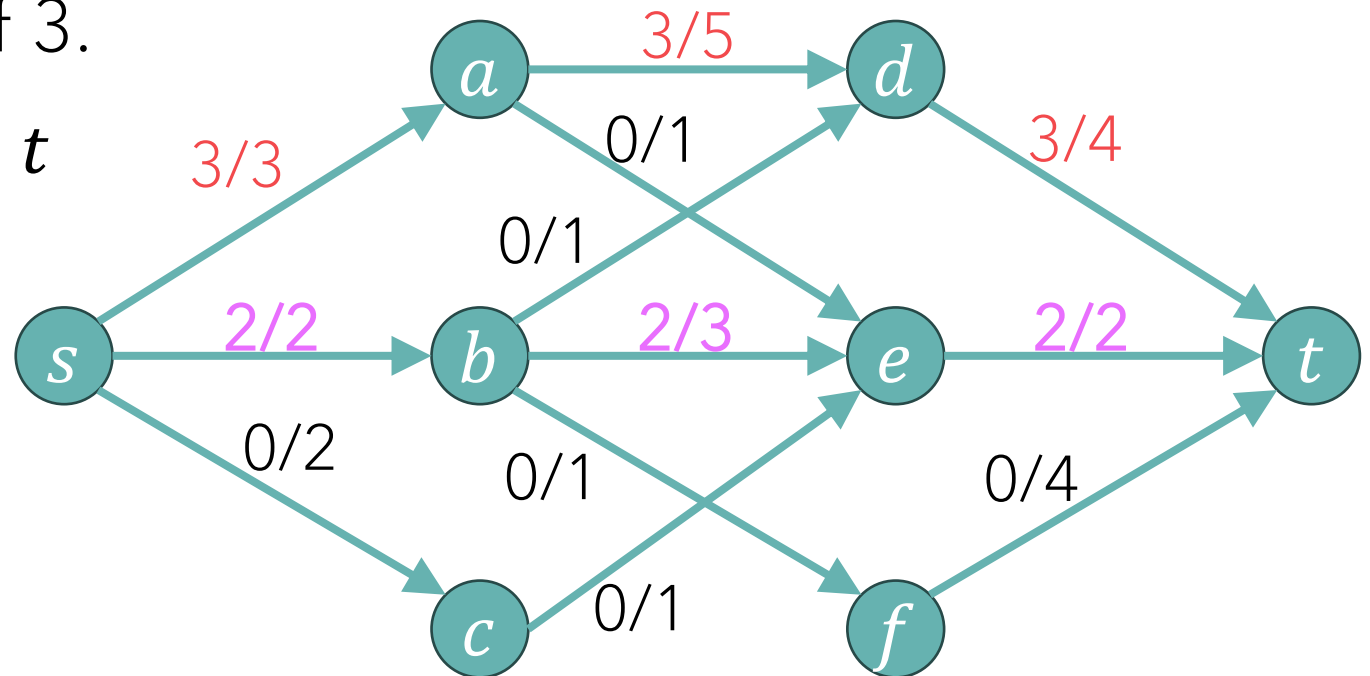
We know every unit of flow must originate at s and end up at t .

What if we greedily select the highest-capacity path from s to t ?

Our first selection would be $s \rightarrow a \rightarrow d \rightarrow t$ with flow of 3.

Next we take $s \rightarrow b \rightarrow e \rightarrow t$ with flow of 2.

Aaaaand... we're done.
There are no $s \rightarrow t$ paths that have any capacity remaining.

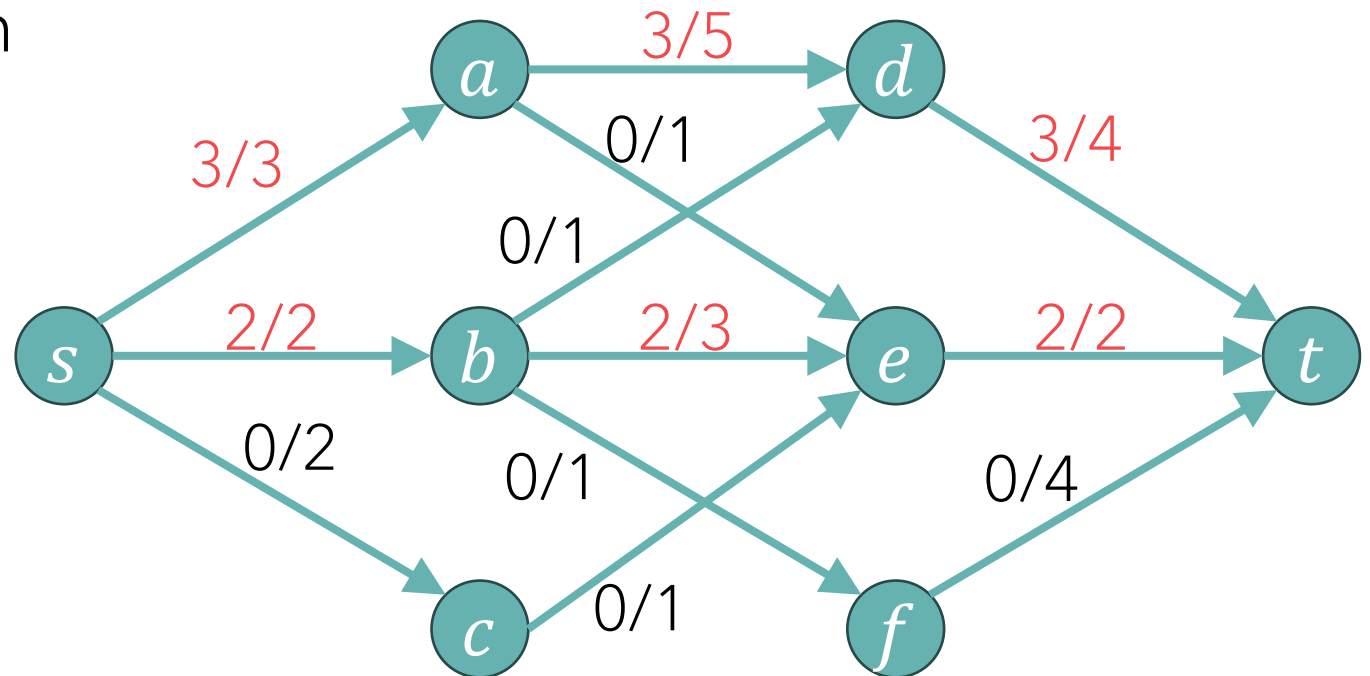


IMPROVING THE GREEDY SOLUTION

Except – this isn't the optimal flow on this network!

We can readily observe that there is spare capacity coming out of s and spare capacity leading into t . Can we use it?

The problem is that, given the current flow, if we try to add more, we'll violate conservation somewhere.



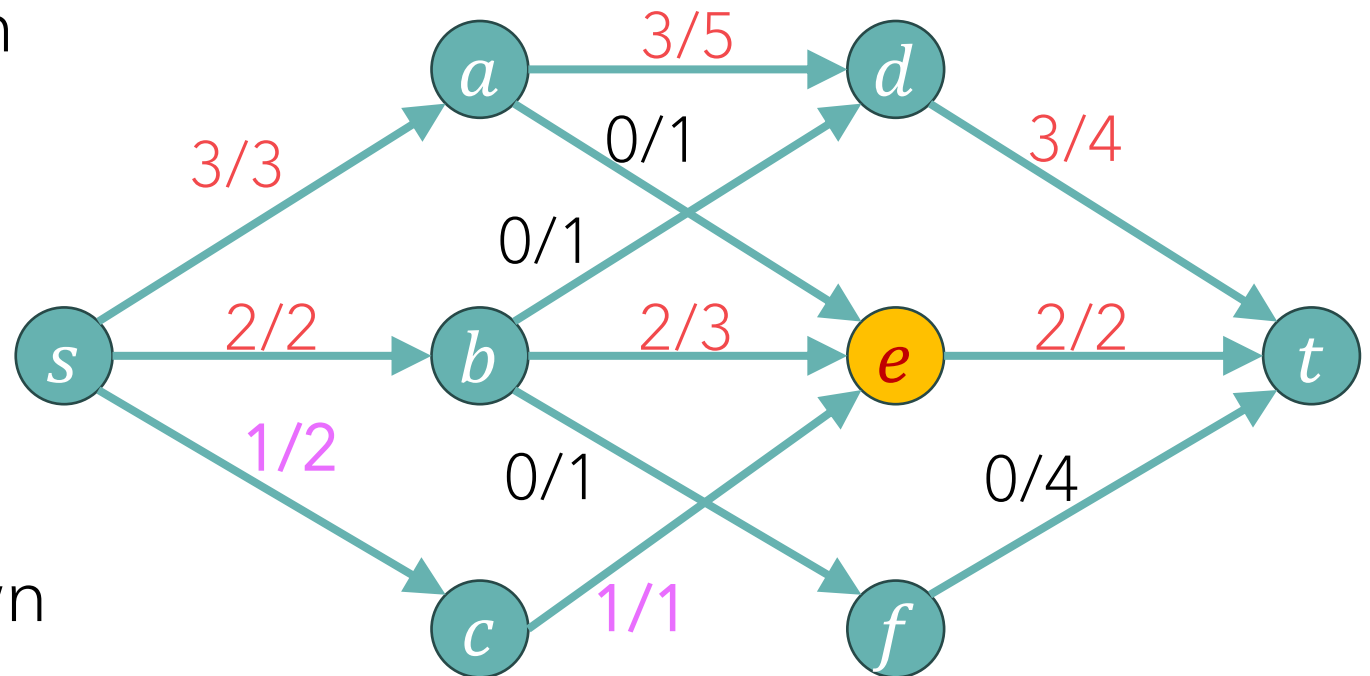
IMPROVING THE GREEDY SOLUTION

Except – this isn't the optimal flow on this network!

We can readily observe that there is spare capacity coming out of s and spare capacity leading into t . Can we use it?

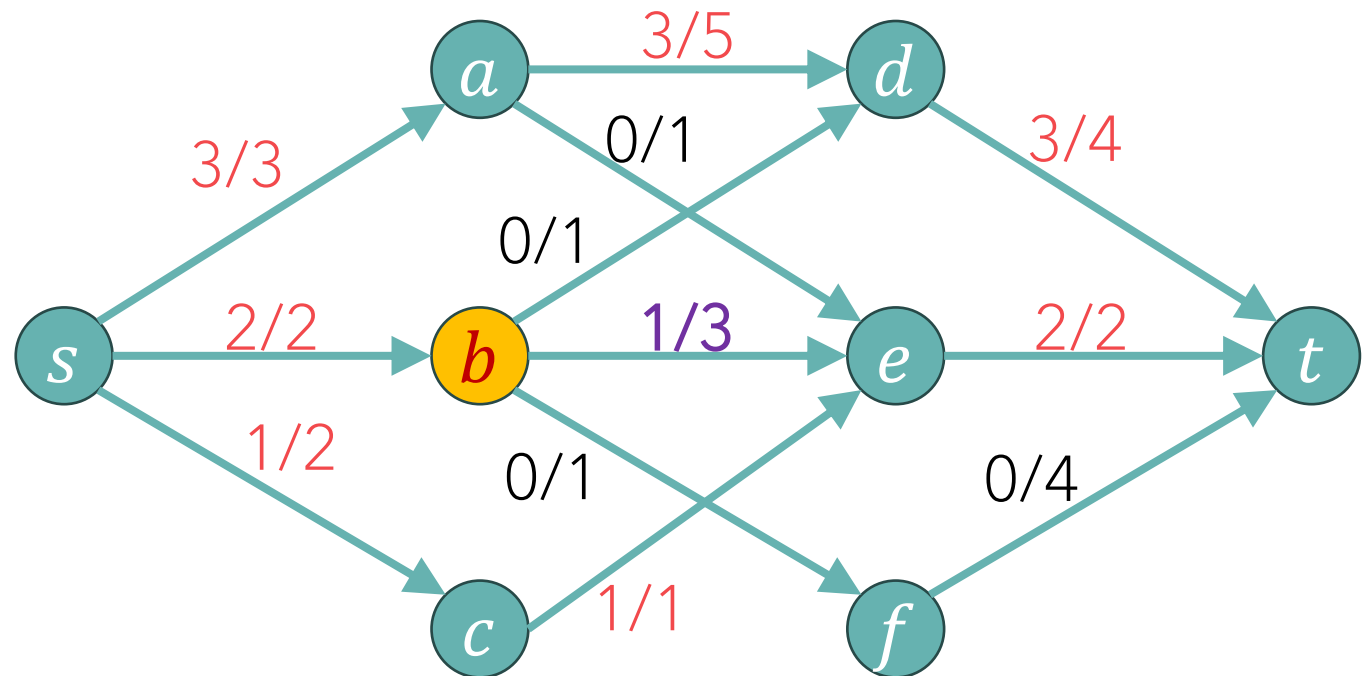
The problem is that, given the current flow, if we try to add more, we'll violate conservation somewhere.

We could reduce another flow into e instead, but this just kicks the can down the road...



IMPROVING THE GREEDY SOLUTION

... but this is actually an improvement, because *b* has available outgoing capacity we can use.

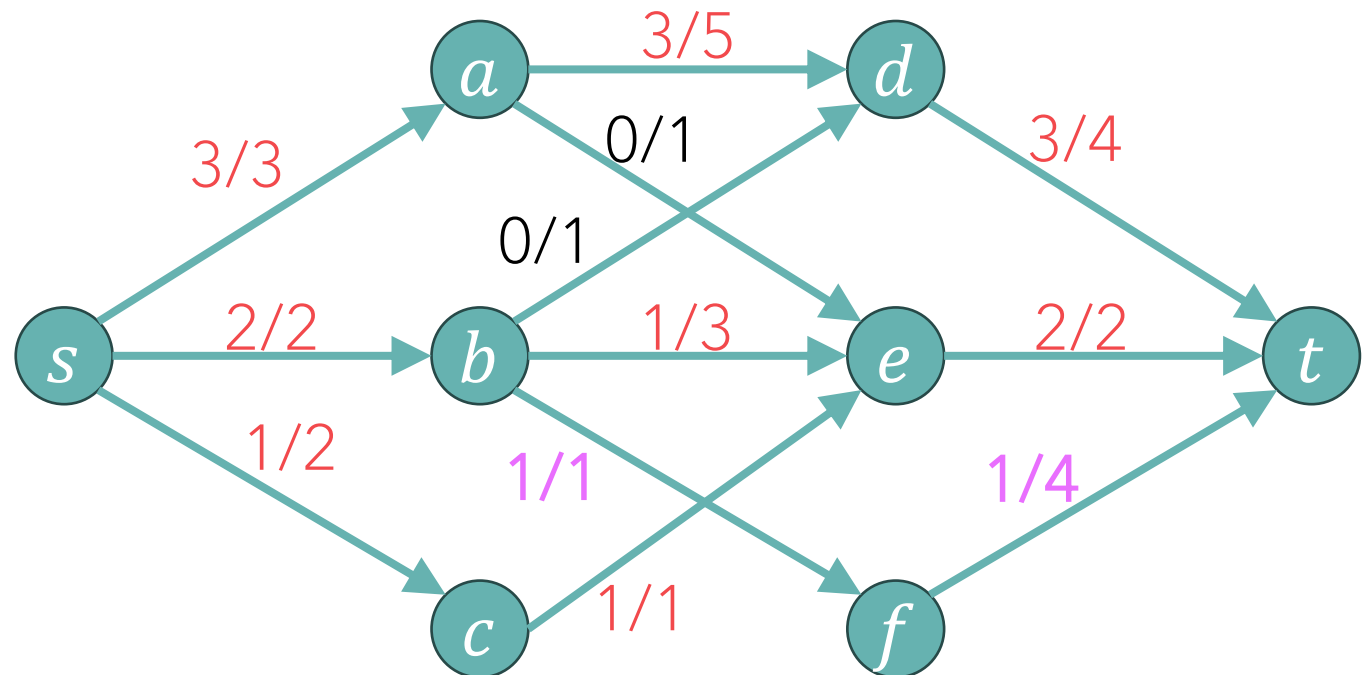


IMPROVING THE GREEDY SOLUTION

... but this is actually an improvement, because *b* has available outgoing capacity we can use.

Here, then, is an optimal flow with value 6. (There are others.)

How do we know that this is the best we can do?



BOTTLENECKS

A truly distressing number of commuters drive from northern NJ into NYC every morning.

Q: What are the choke points in this traffic network?

A: Bridges and tunnels



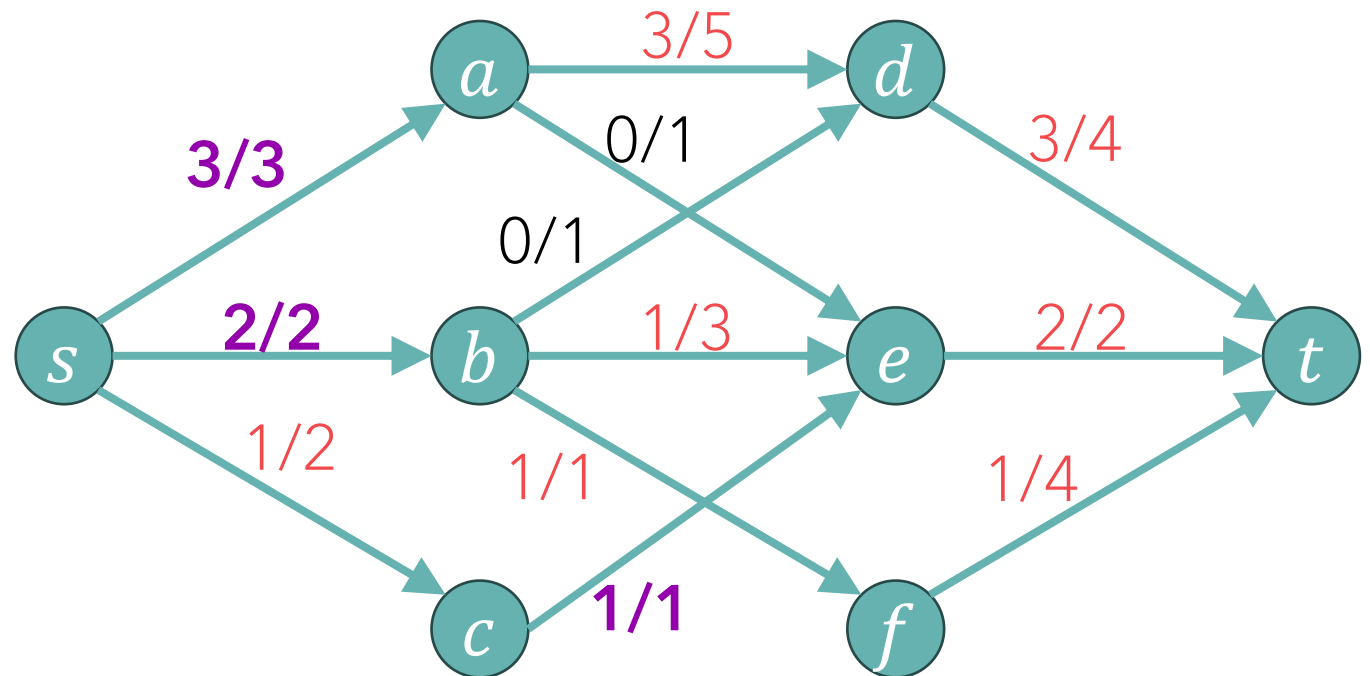
IMPROVING THE GREEDY SOLUTION

Here, then, is an optimal flow with value 6. (There are others.)

How do we know that this is the best we can do?

Consider the three boldfaced edges. Each of them is *saturated* - at maximum capacity.

Further, every $s \rightarrow t$ path must use one of these three edges.



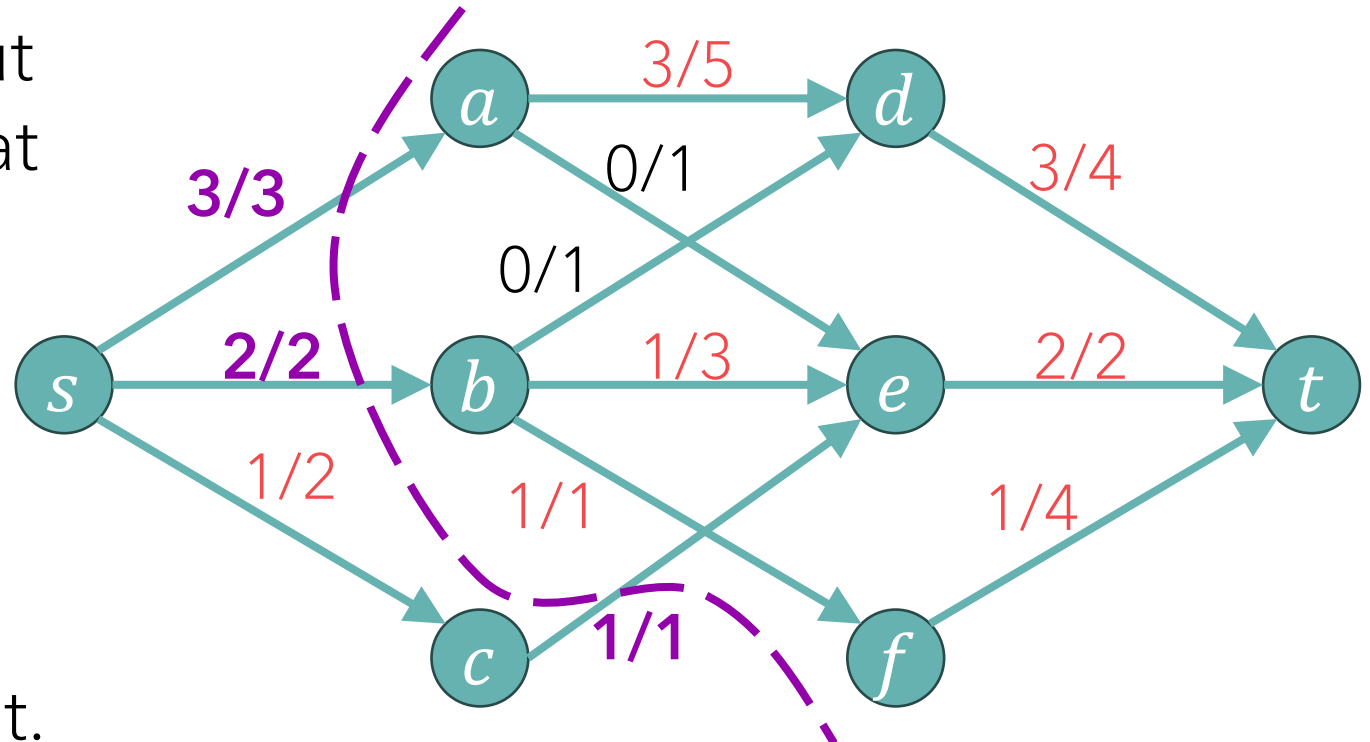
THE CUT

A *cut* of a graph is a partition of the vertices into two groups.

For flow networks, s - t cuts are of particular interest – these force the source and sink to opposite sides of the cut.

We have located an s - t cut that is fully saturated. What does that tell us?

Claim 1: Any flow that has a fully saturated s - t cut is maximal, with value equal to the sum of the flows across that cut.

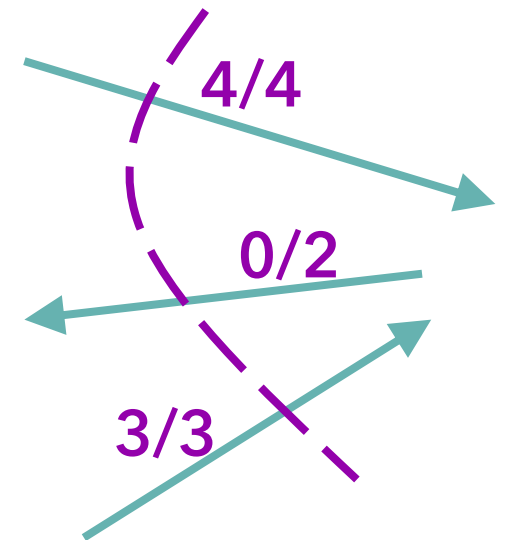
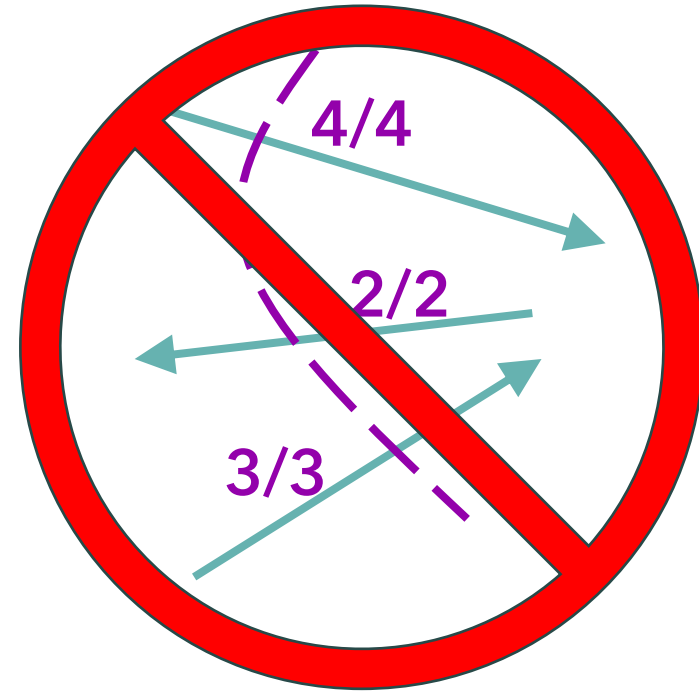


SATURATED CUT $\Rightarrow$ MAX FLOW

First, a clarification: A fully saturated s - t cut cannot have any backflow.

An s - t cut is fully saturated when every edge leading from the source side to the sink side has flow equal to its capacity, and every edge from the sink side to the source side has 0 flow.

Given that definition, the previous claim is obvious. Every $s \rightarrow t$ path uses one of the forward edges of the cut. We can't possibly have more flow than crosses the cut. And since there's no backflow, everything crossing the cut originates at s .



MINIMUM CUTS (I.E. BOTTLENECKS)

In fact, we can make a stronger statement.

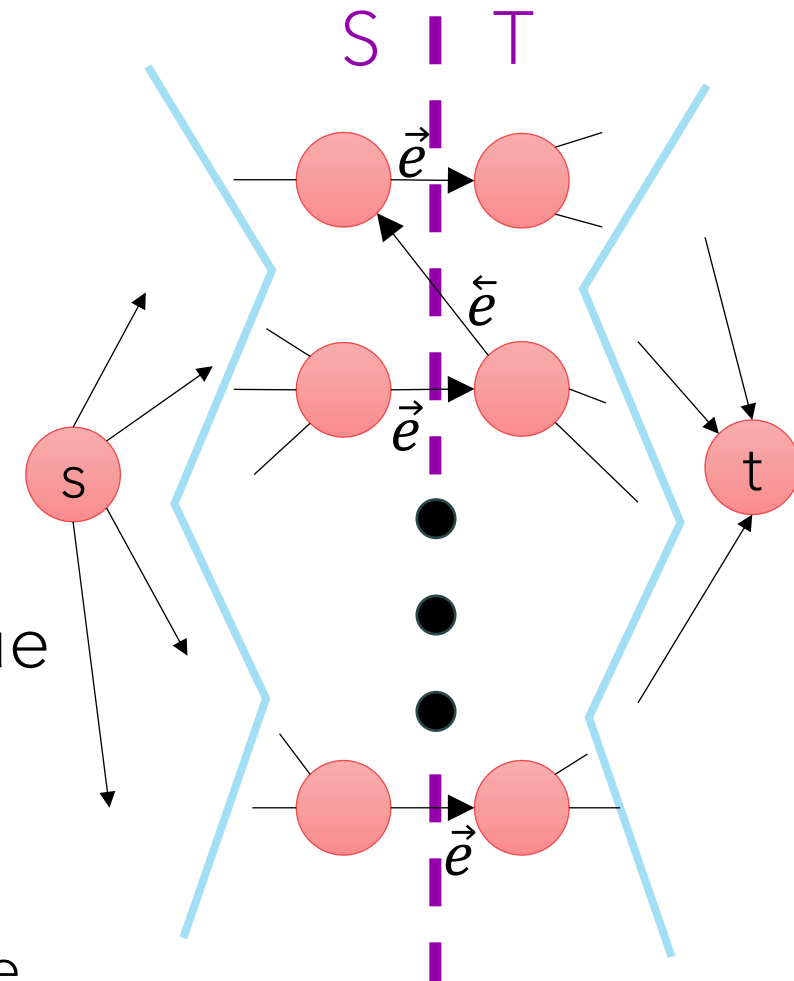
Let (S, T) be any s - t cut: $S, T \subseteq V, s \in S, t \in T$, and $S \cap T = \emptyset$. Define a *forward edge* as $\vec{e} = (u, v)$ such that $u \in S$ and $v \in T$, and a *backward edge* $\tilde{e} = (u, v)$ with $u \in T, v \in S$.

Let the *capacity of the cut* $c(S, T) = \sum c(\vec{e})$.

Claim 2: Let f be any valid flow. Then the value of the flow $v(f)$ will be $\sum f(\vec{e}) - \sum f(\tilde{e})$.

Claim 3: $v(f) \leq c(S, T)$

In other words, no valid flow can ever have a value that exceeds the capacity of any s - t cut.



MINCUT: IDENTIFYING THE BOTTLENECKS

Finding the s - t cut with minimum capacity feels like an important problem now – it gives us an upper bound on the maximum flow. This is the *Minimum Capacity Cut* problem, or *MinCut* for short.

The link between MaxFlow and MinCut is even tighter than we have stated so far: Observe that our bound means that if we can find a flow f that fully saturates a cut, then we have solved both problems at once: By the bound in Claim 3, there can't be a smaller cut than $v(f)$ (since a flow with value this large exists), nor can we have a larger flow (since a cut with capacity this small exists).

ENTER FORD AND FULKERSON

Once again, we turn to the RAND Corporation for an algorithm to solve our problem – unsurprising, as network flow became of interest because of military logistics.

Lester Ford Jr. (yes, as in Bellman-Ford) and Delbert Fulkerson both were working at RAND in the 1950s when they created an internal technical report containing their procedure for solving *MaxFlow*; two years later, they [published a paper](#) in the *Canadian Journal of Mathematics*.

Given a flow network, the Ford-Fulkerson procedure always finds a flow that fully saturates an s - t cut; as discussed on the previous slide, this is an optimal solution to both *MaxFlow* and *MinCut*.

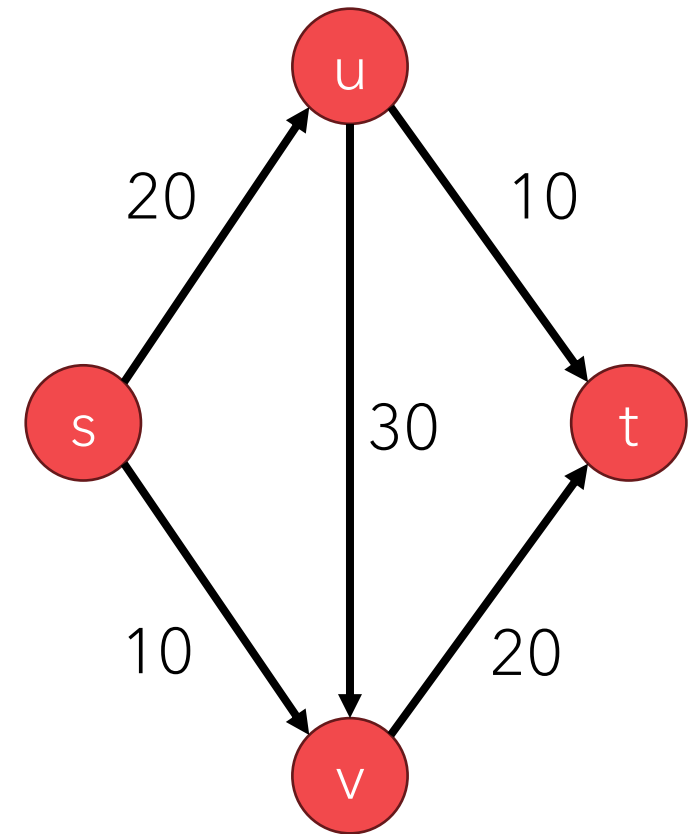
THE FORD-FULKERSON PROCEDURE

The key idea (which we have already seen) is that, given an existing flow, we may need to push some of that flow back to take a different path.

We can model this idea by creating a *residual graph*, which contains reversed edges representing actual flow.

Consider the example at right.

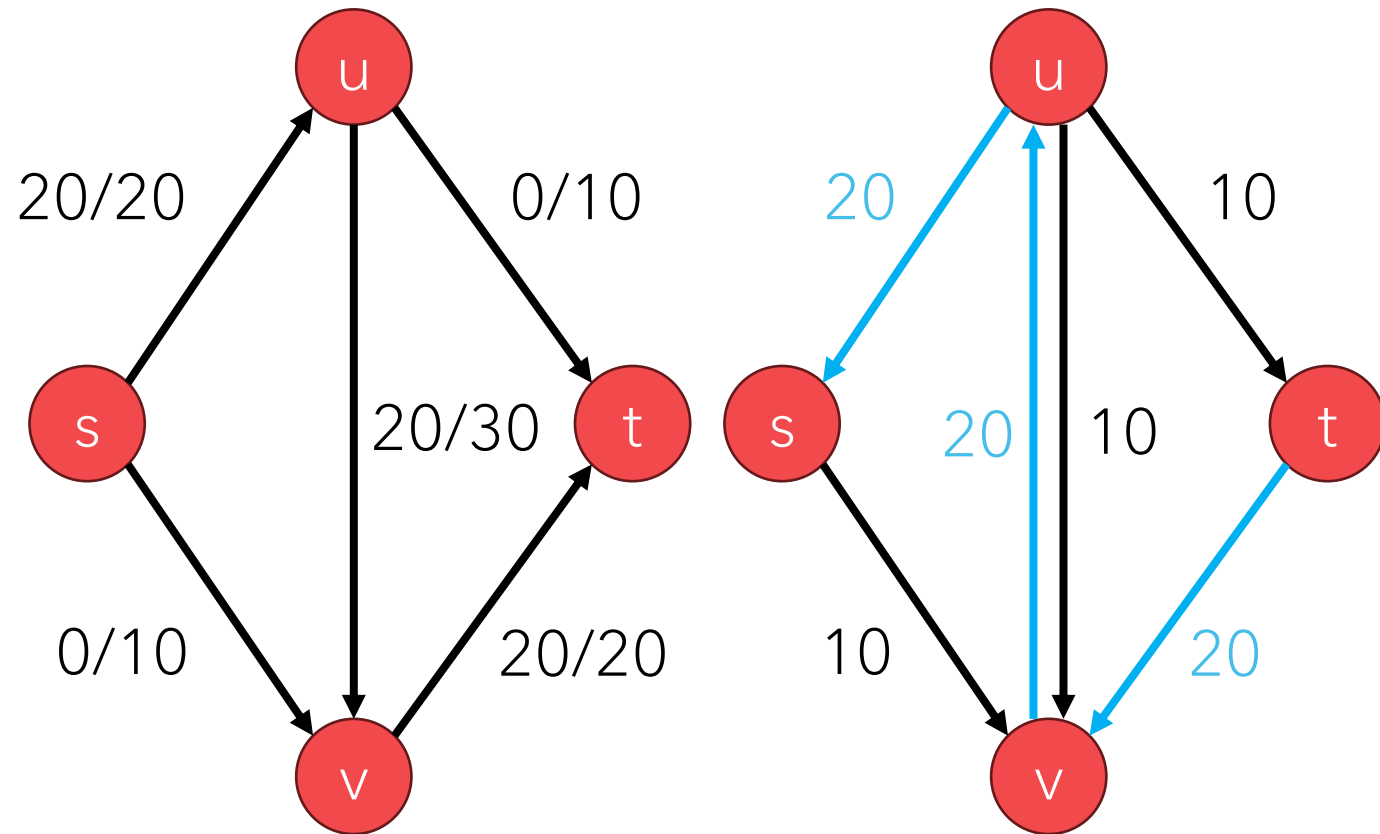
We might start, as before, by taking the maximum-capacity path: $s \rightarrow u \rightarrow v \rightarrow t$.



THE FORD-FULKERSON PROCEDURE

Once we have added flow we then create the residual graph.

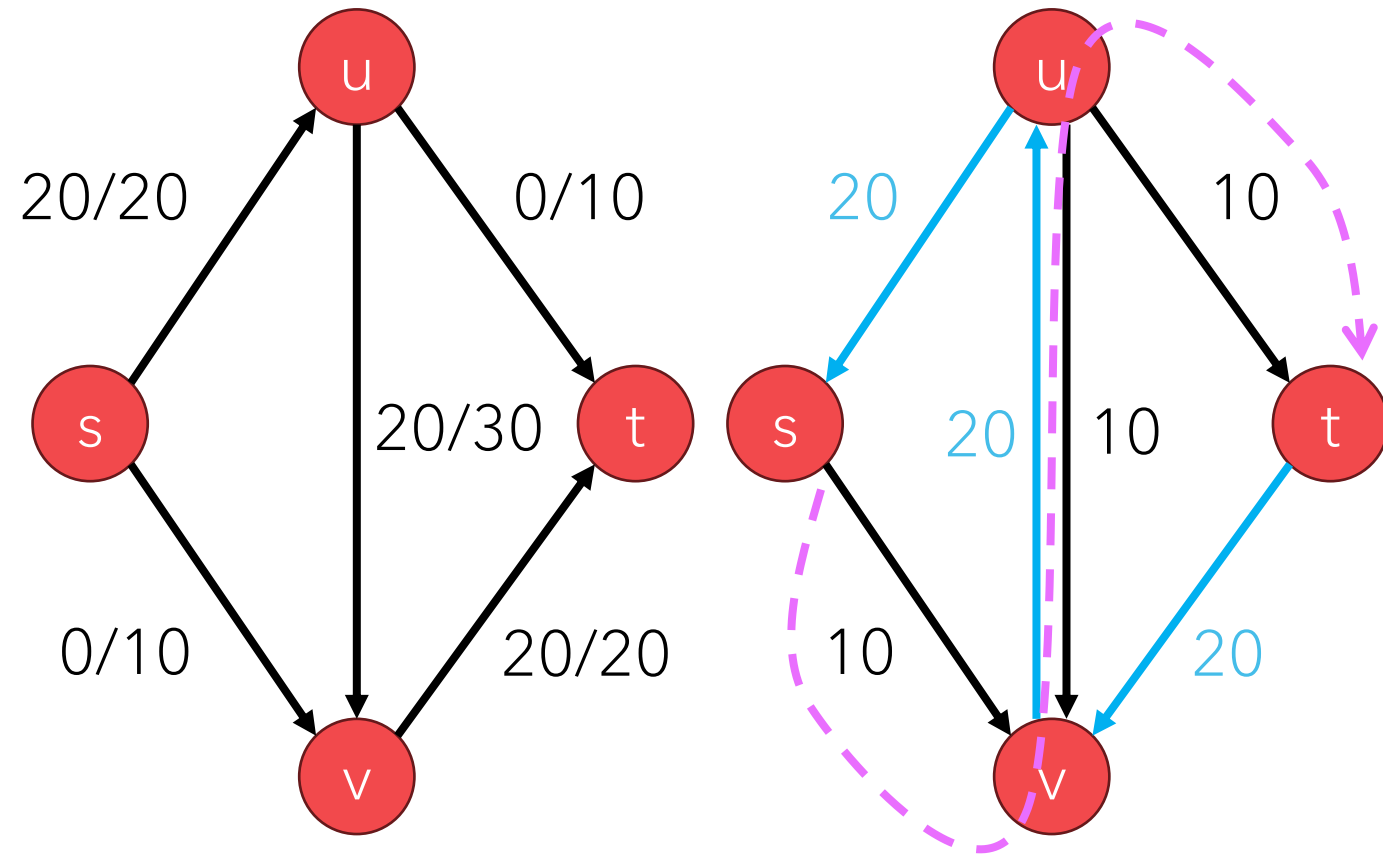
Again, the residual graph has reversed edges everywhere that has actual flow and normal edges where we still have real capacity available.



THE FORD-FULKERSON PROCEDURE

We then look for $s \rightarrow t$ paths in the residual graph; these are called *augmenting paths*.

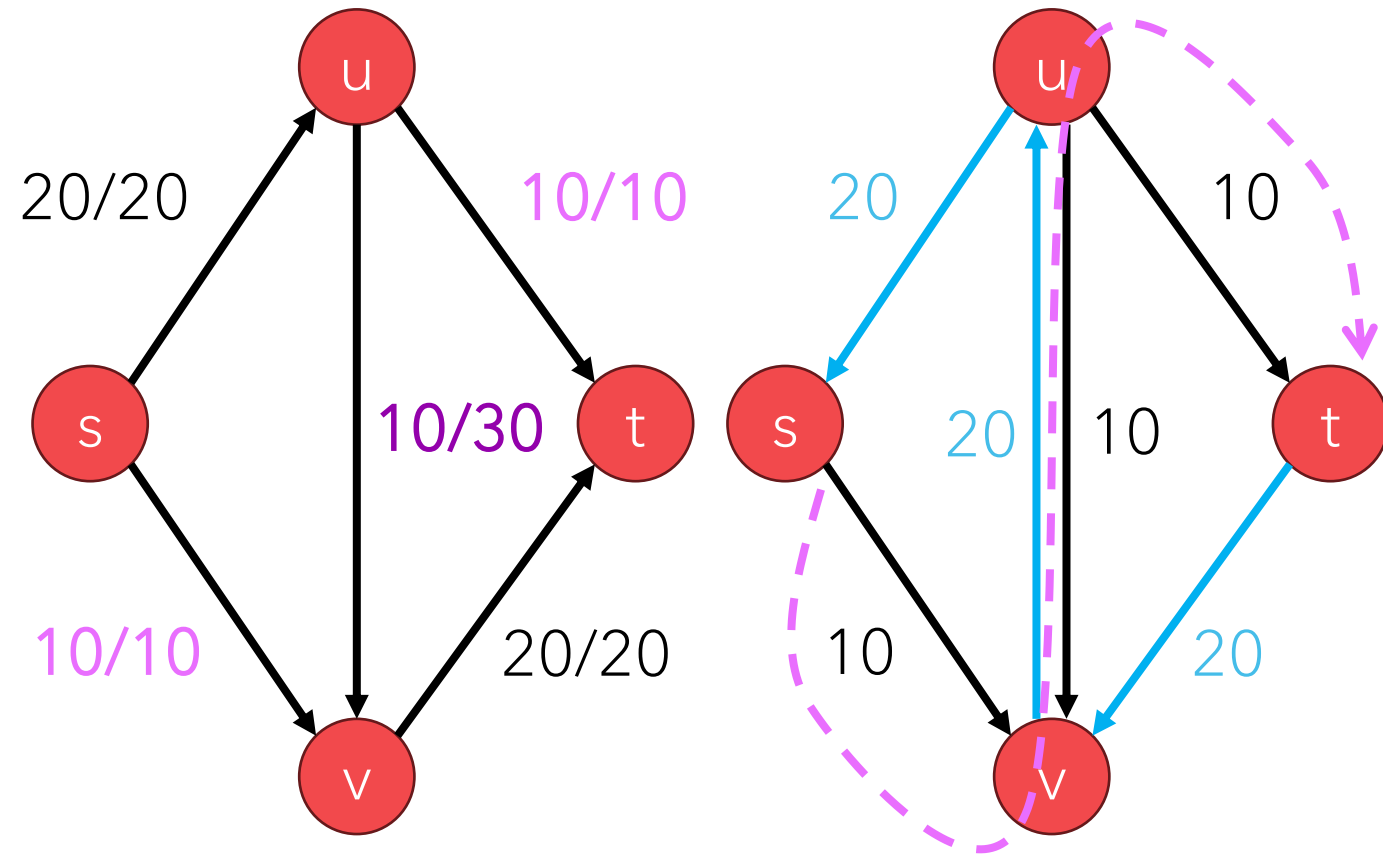
This augmenting path represents adding flow from s to v , undoing some of the flow from u to v , and adding flow from u to t .



THE FORD-FULKERSON PROCEDURE

We then look for $s \rightarrow t$ paths in the residual graph; these are called *augmenting paths*.

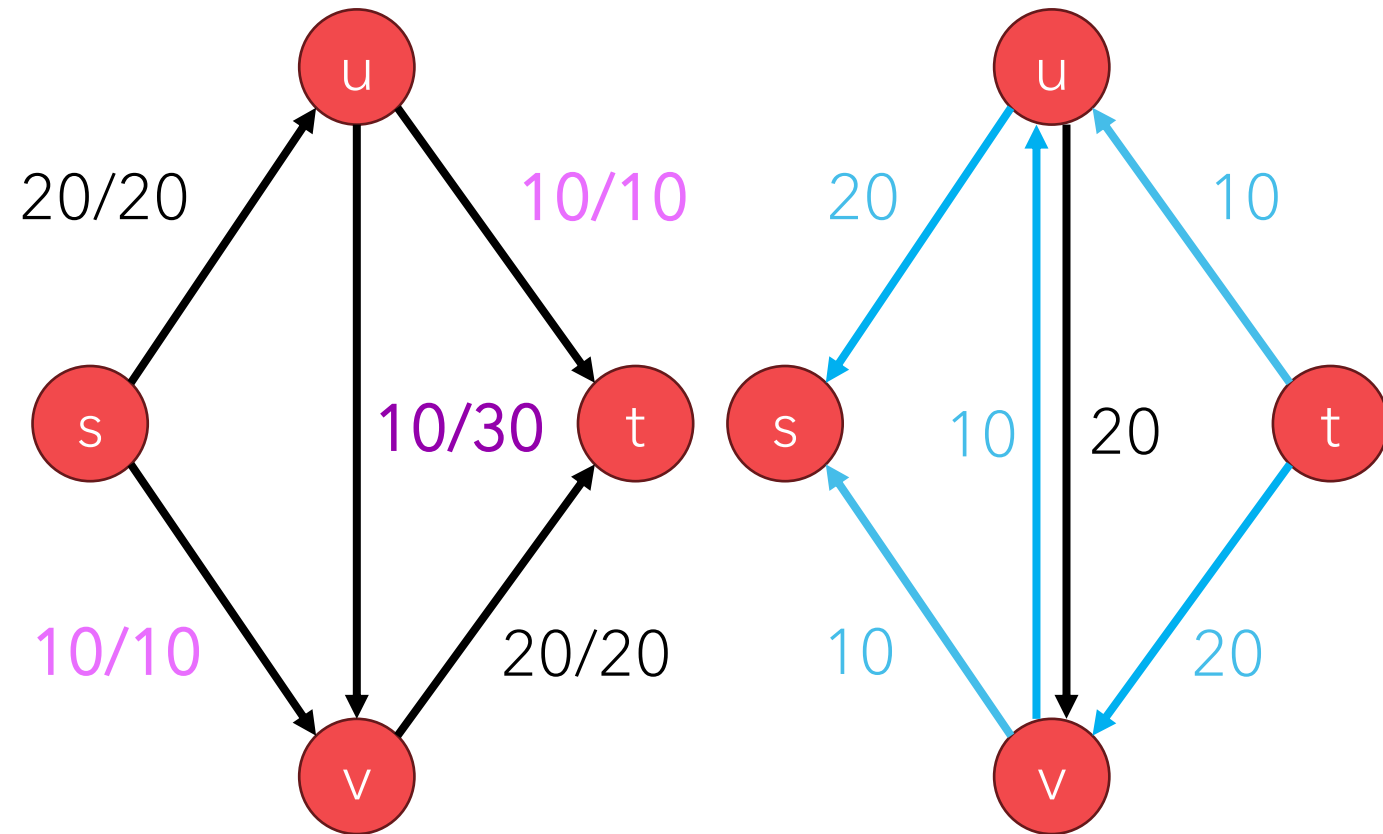
This augmenting path represents adding flow from s to v , undoing some of the flow from u to v , and adding flow from u to t .



THE FORD-FULKERSON PROCEDURE

The new residual graph now looks like this.

There are no more $s \rightarrow t$ paths in the residual graph, so we are done.



THE AUGMENT SUBROUTINE

Assume that we are given some flow f and have already found an augmenting path P through the residual graph R_f . Then:

```
def augment(f,P):  
    f' = f  
    let b = min(c(e)) of all e ∈ P  
    for e in P:  
        if e is a normal edge:  
            f'(e) = f(e) + b  
        else: # e is a reversed edge  
            f'(e) = f(e) - b  
    return f'
```

CORRECTNESS OF AUGMENT

f' always obeys the capacity and direction conditions because b is always chosen to be the minimum capacity of the augmenting path.

f' also obeys the conservation conditions, because for every node v on the path (other than s and t):

- Incoming edges increase net flow by b (normal edges as usual; reverse edges by decreasing outflow).
- Outgoing edges decrease net flow by b (normal edges as usual; reverse edges by decreasing inflow).
- Every intermediate vertex on the path has one of each.

```
def augment(f,P):  
    f' = f  
    let b = min(c(e)) of all e ∈ P  
    for e in P:  
        if e is a normal edge:  
            f'(e) = f(e) + b  
        else: # e is a reversed edge  
            f'(e) = f(e) - b  
    return f'
```

THE FORD-FULKERSON PROCEDURE

```
def maxFlow(G,s,t):  
    f(e) = 0 for all e  
    Rf = G  
    while Rf has an s-t path with capacity > 0:  
        Let P be any such path  
        f' = augment(f,P)  
        f = f'  
        update Rf based upon the new flow  
    return f
```

Key questions: How can we guarantee that this returns the maximum flow? Are we even sure this terminates?

ANALYSIS OF FFP

We are guaranteed termination.

- $v(f)$ can't be higher than C , the capacity of the minimum cut.
- $v(f)$ starts at 0 and increases each time through the loop.
- Because the capacities are integers, that increase will also be an integer.

```
def maxFlow(G,s,t):  
    f(e) = 0 for all e  
    Rf = G  
    while Rf has an s-t path with cap > 0:  
        Let P be any such path  
        f' = augment(f,P)  
        f = f'  
        update Rf based upon the new flow  
    return f
```

The procedure also always produces a fully saturated s - t cut.

ANALYSIS OF FFP

Consider the final instance of R_f . Let S^* be the set of all nodes reachable from s in R_f , and let $T^* = V - S^*$.

Because the **while** loop terminated, $t \in T^*$. Thus, (S^*, T^*) is an s - t cut.

- Let $\vec{e} = (u, v)$ be an edge of G with $u \in S^*$ and $v \in T^*$. Then $f(\vec{e}) = c(\vec{e})$; if not, there would be an edge from u to v in R_f , and v would be reachable from s via u . Then v would be in S^* and not T^* .
- Let $\tilde{e} = (u, v)$ be an edge of G with $u \in T^*$ and $v \in S^*$. Then $f(\tilde{e}) = 0$; if not, there would be a reversed edge from v to u in R_f , and u would be reachable from s via v . Then u would be in S^* and not T^* .

(S^*, T^*) is thus fully saturated. By our previous arguments, it is a minimum s - t cut, and its capacity is the maximal flow value. ■

```
def maxFlow(G, s, t):  
    f(e) = 0 for all e  
    Rf = G  
    while Rf has an s-t path with cap > 0:  
        Let P be any such path  
        f' = augment(f, P)  
        f = f'  
        update Rf based upon the new flow  
    return f
```

RUNTIME OF FFP

```
def maxFlow(G,s,t):  
    f(e) = 0 for all e  
    Rf = G  
    while Rf has an s-t path with cap > 0:  
        Let P be any such path  
        f' = augment(f,P)  
        f = f'  
        update Rf based upon the new flow  
    return f
```

What is the runtime of this algorithm?
Answer now in the Submittity poll.



RUNTIME OF FFP

The while loop executes a maximum of C times.

We don't know C up front, but we can bound it above with $C_s = \sum_{v \in V} c(s, v)$, the sum of the capacities of the edges leading out from s .

augment requires $O(E)$ time, so the overall runtime is $O(CE)$. Note that this is another pseudopolynomial runtime.

It also misses the elephant in the room: **How do we select a path?**

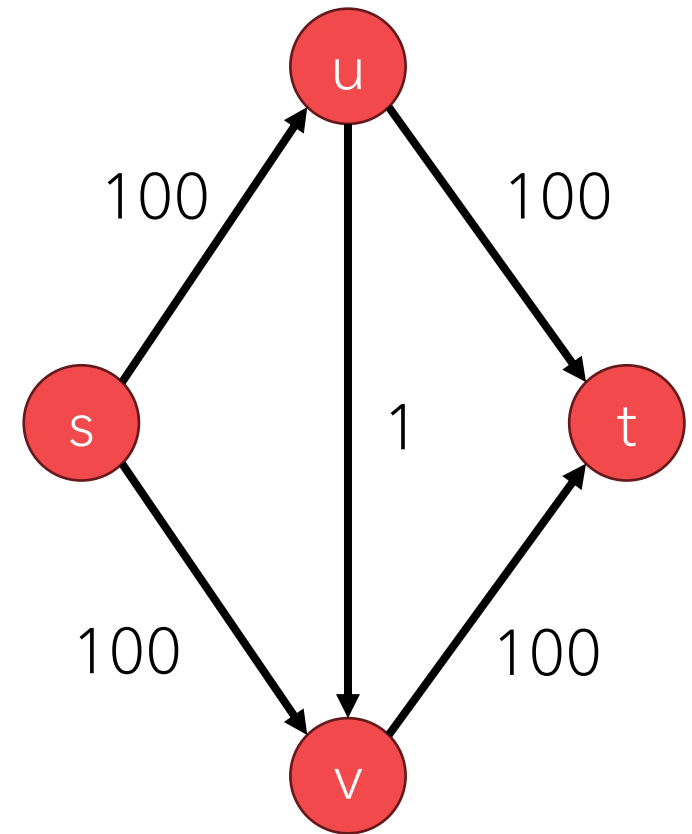
```
def maxFlow(G,s,t):  
    f(e) = 0 for all e  
    Rf = G  
    while Rf has an s-t path with cap > 0:  
        Let P be any such path  
        f' = augment(f,P)  
        f = f'  
        update Rf based upon the new flow  
    return f
```

PATH(OLOGICAL) EXAMPLES

The baseline FFP doesn't actually tell us how to select a path. (This is why I haven't referred to it as an algorithm.)

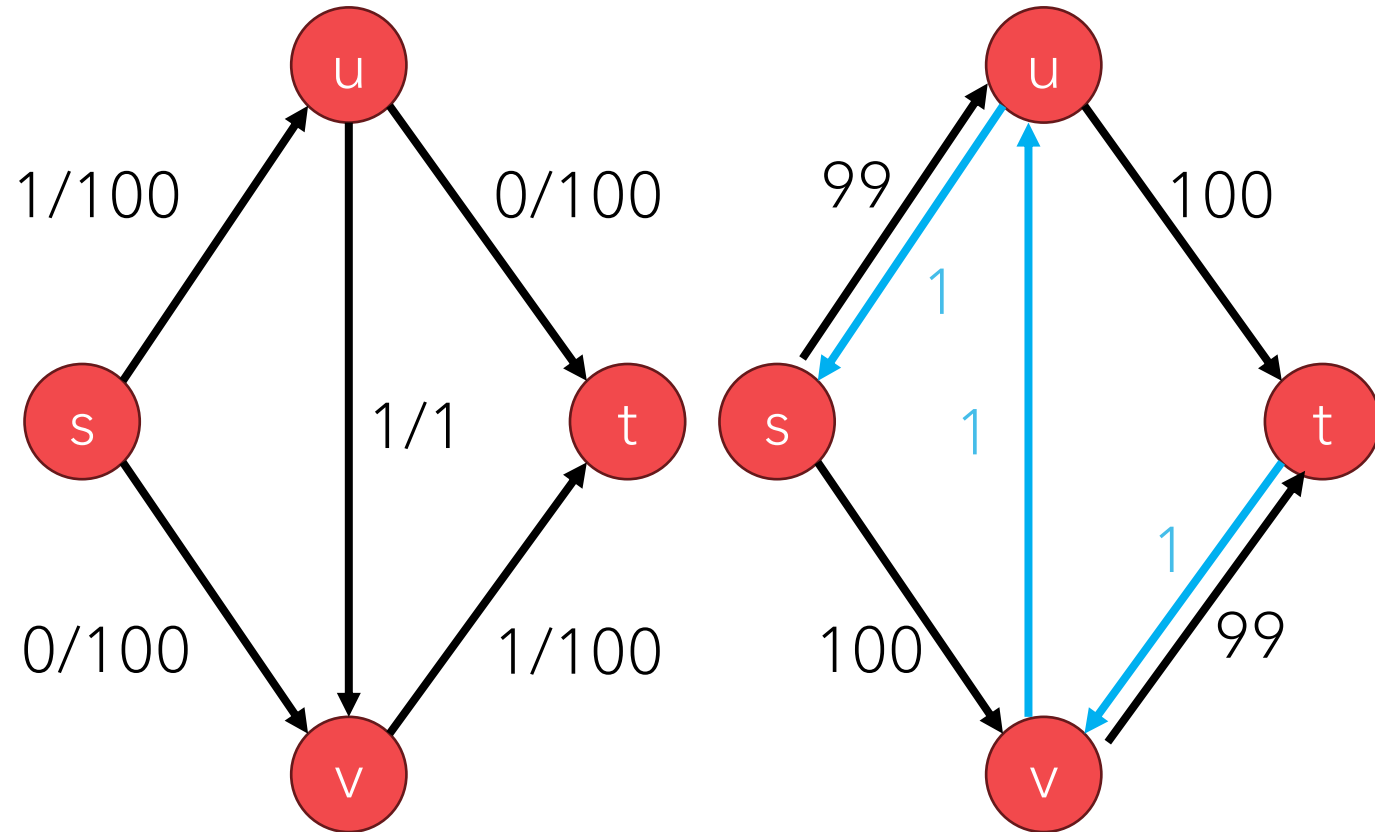
With a poor path selection scheme, FFP can go through a stupid number of iterations.

In this example, we might pick the augmenting path $s \rightarrow u \rightarrow v \rightarrow t$ with capacity 1.



PATH(OLOGICAL) EXAMPLES

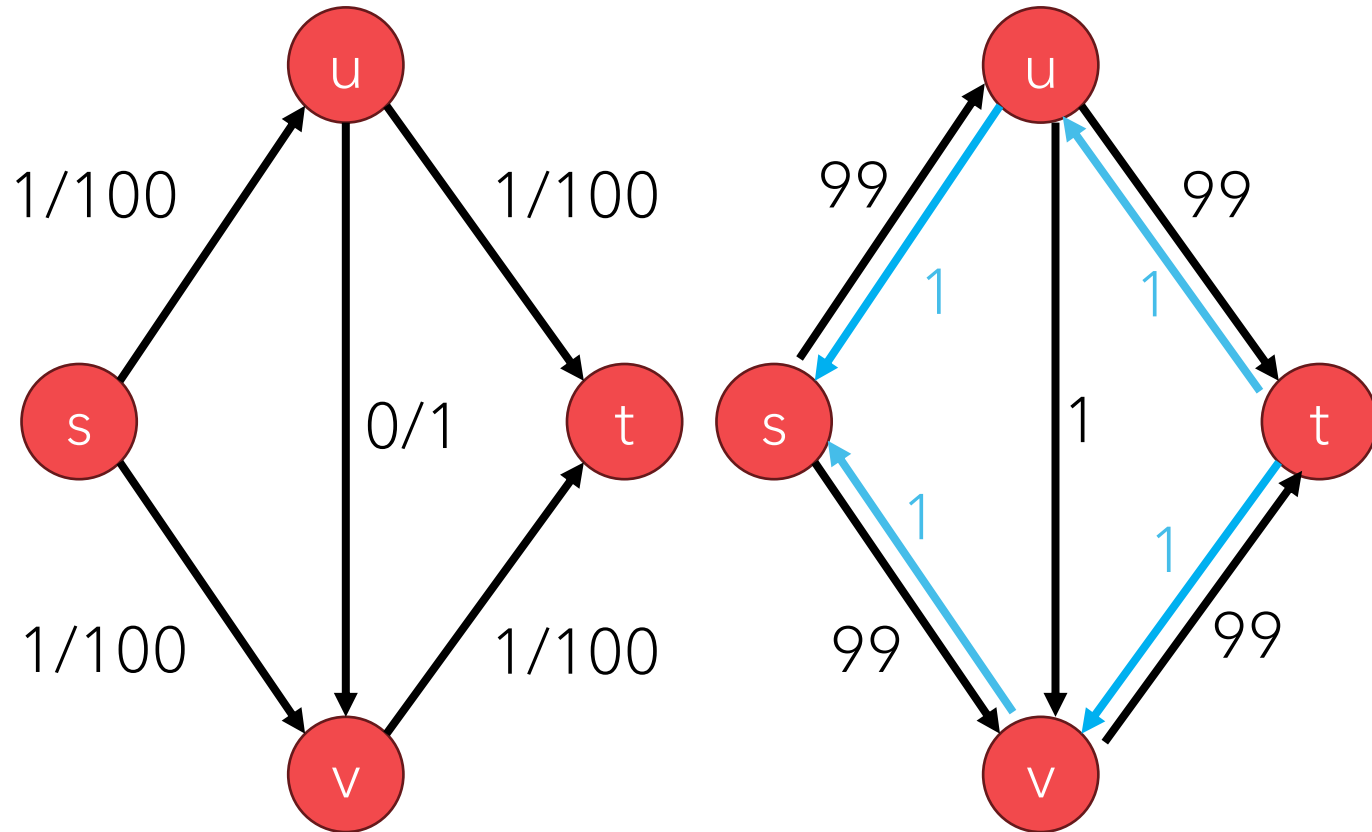
We might then select the augmenting path $s \rightarrow v \rightarrow u \rightarrow t$, also with capacity 1.



PATH(OLOGICAL) EXAMPLES

We might then select the augmenting path $s \rightarrow v \rightarrow u \rightarrow t$, also with capacity 1.

... and then repeat those two steps 99 more times.



FIXING FFP

Yefim Dinitz published a fix for FFP in 1970.

Jack Edmonds and Richard Karp published essentially the same fix in 1972.

So naturally, we now call it the Edmonds-Karp algorithm.

```
def maxFlow(G,s,t):  
    f(e) = 0 for all e  
    Rf = G  
    while Rf has an s-t path with cap > 0:  
        Let P be any such path  
        f' = augment(f,P)  
        f = f'  
        update Rf based upon the new flow  
    return f
```

FIXING FFP

Yefim Dinitz published a fix for FFP in 1970.

Jack Edmonds and Richard Karp published essentially the same fix in 1972.

So naturally, we now call it the Edmonds-Karp algorithm.

The fix itself is actually really simple: Always pick the augmenting path that uses the fewest edges. (Find this using BFS.)

This also brings FFP squarely into polynomial time: It can be proven that the while loop executes no more than $O(VE)$ times, making the overall runtime $O(VE^2)$.

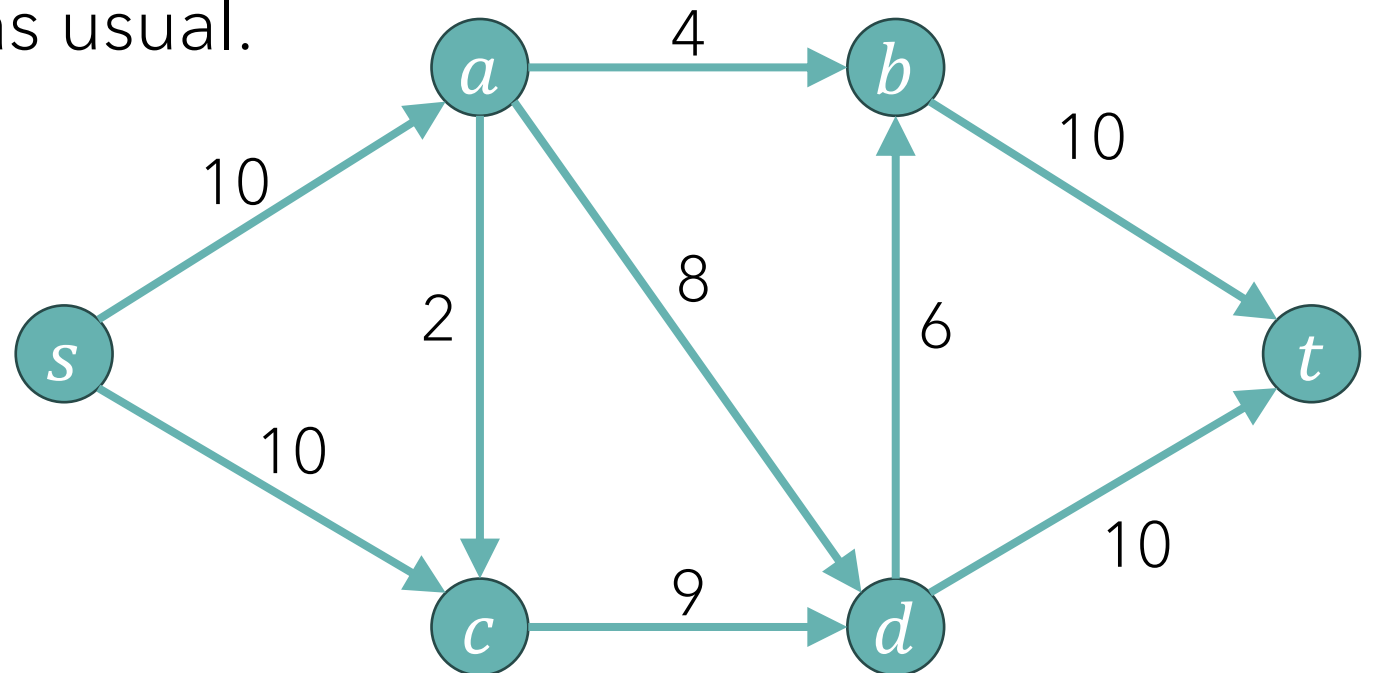
```
def maxFlow(G,s,t):  
    f(e) = 0 for all e  
    Rf = G  
    while Rf has an s-t path with cap > 0:  
        Let P be the shortest such path  
        f' = augment(f,P)  
        f = f'  
        update Rf based upon the new flow  
    return f
```

PRACTICE WITH EDMONDS-KARP

Let's take a few minutes to try the algorithm now.

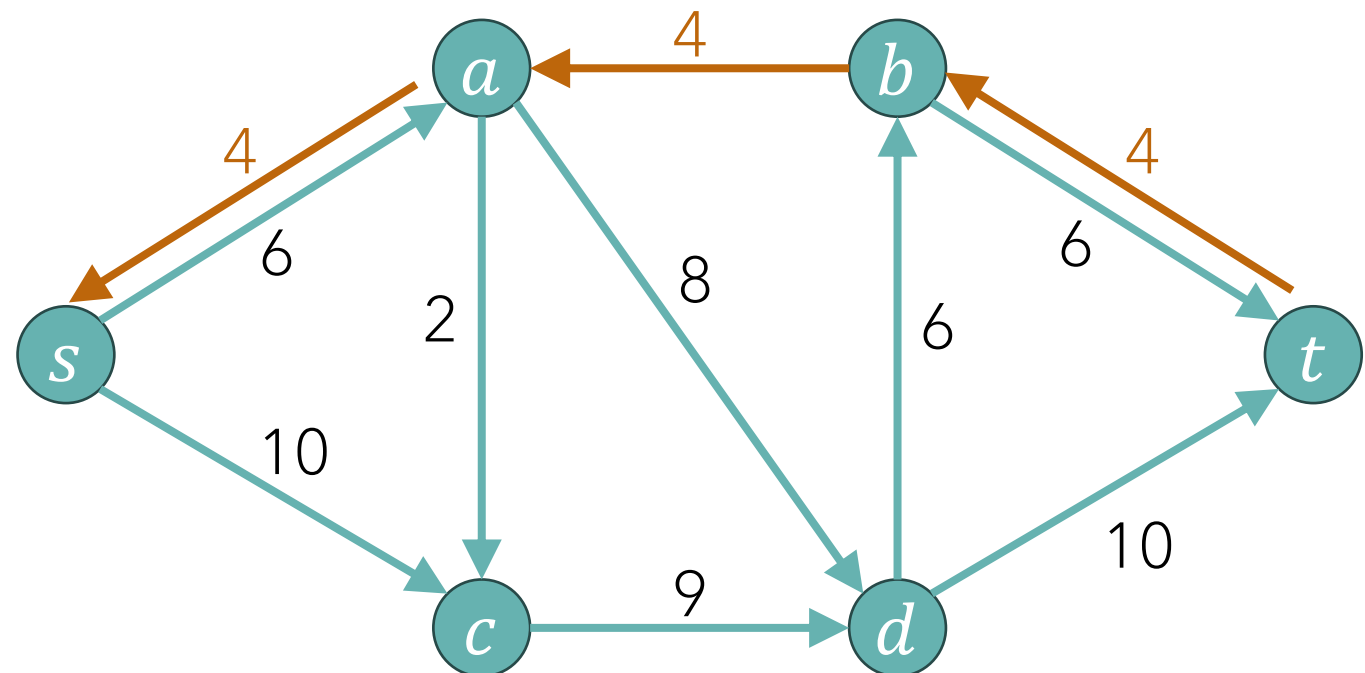
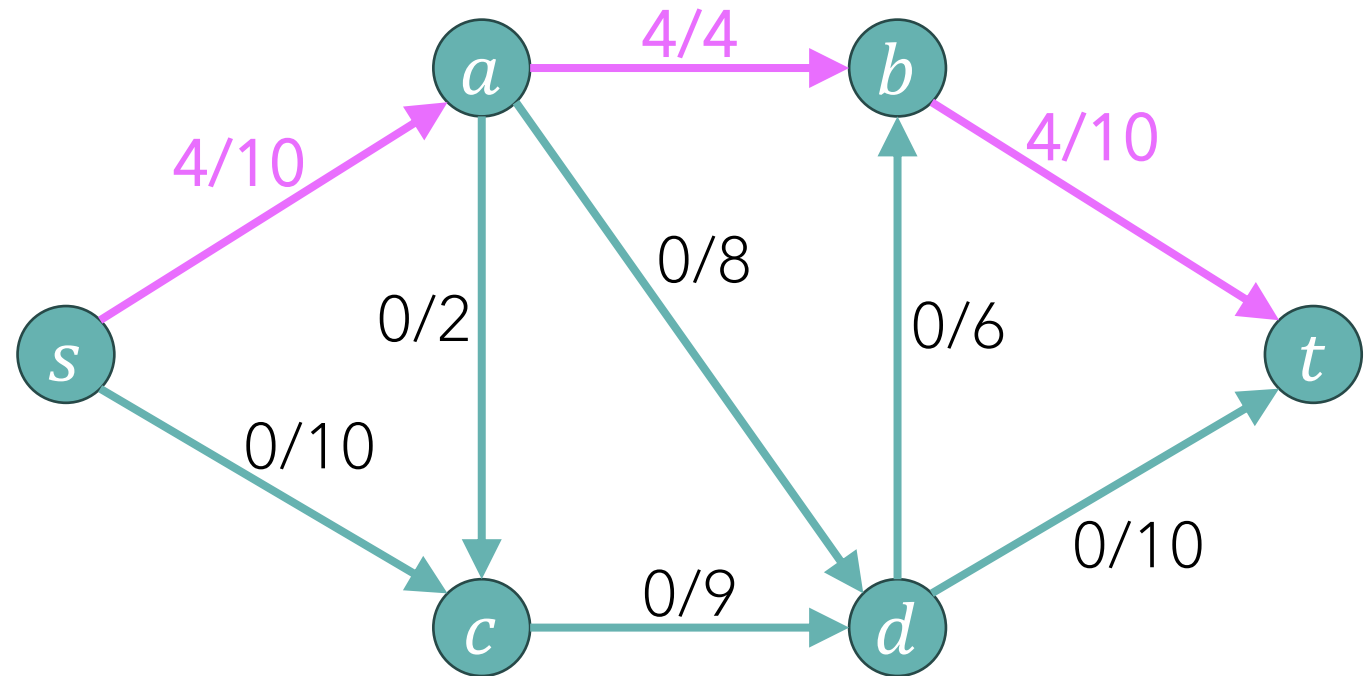
Make sure to draw the residual graph as part of your process – you will surely be asked to do this again at some point.

Break ties alphabetically as usual.



EDMONDS-KARP

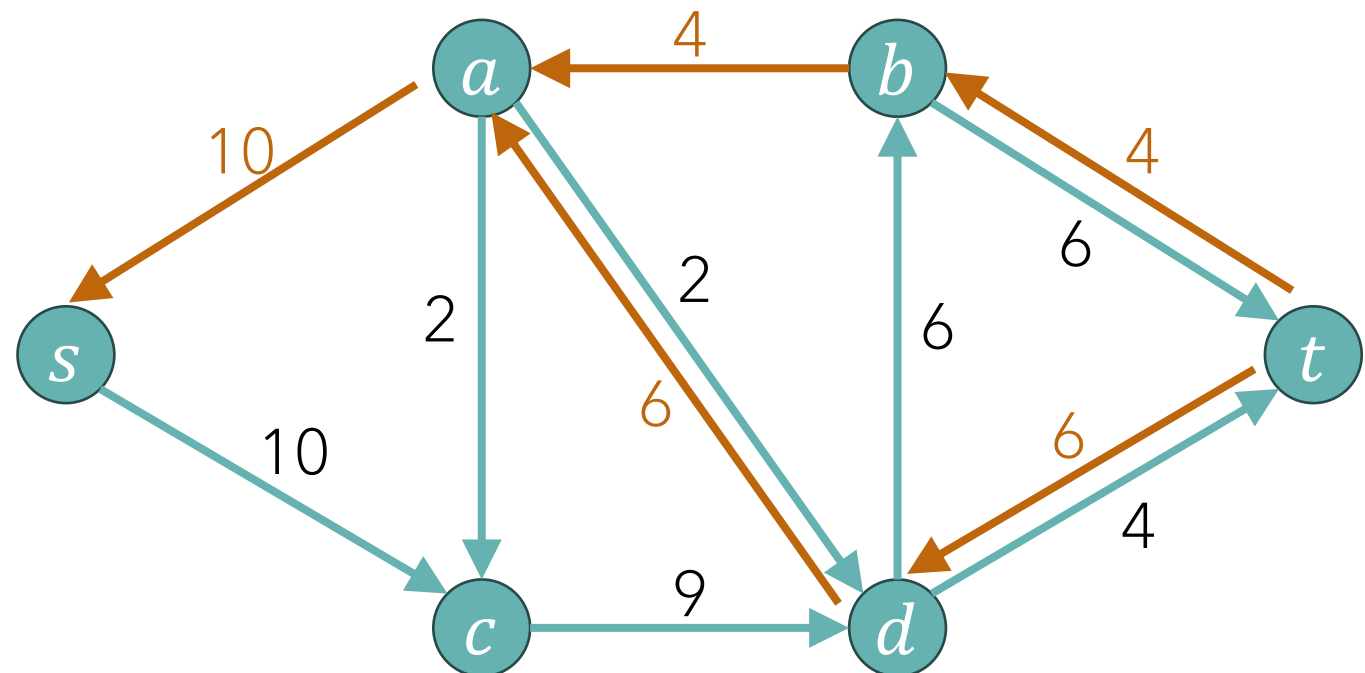
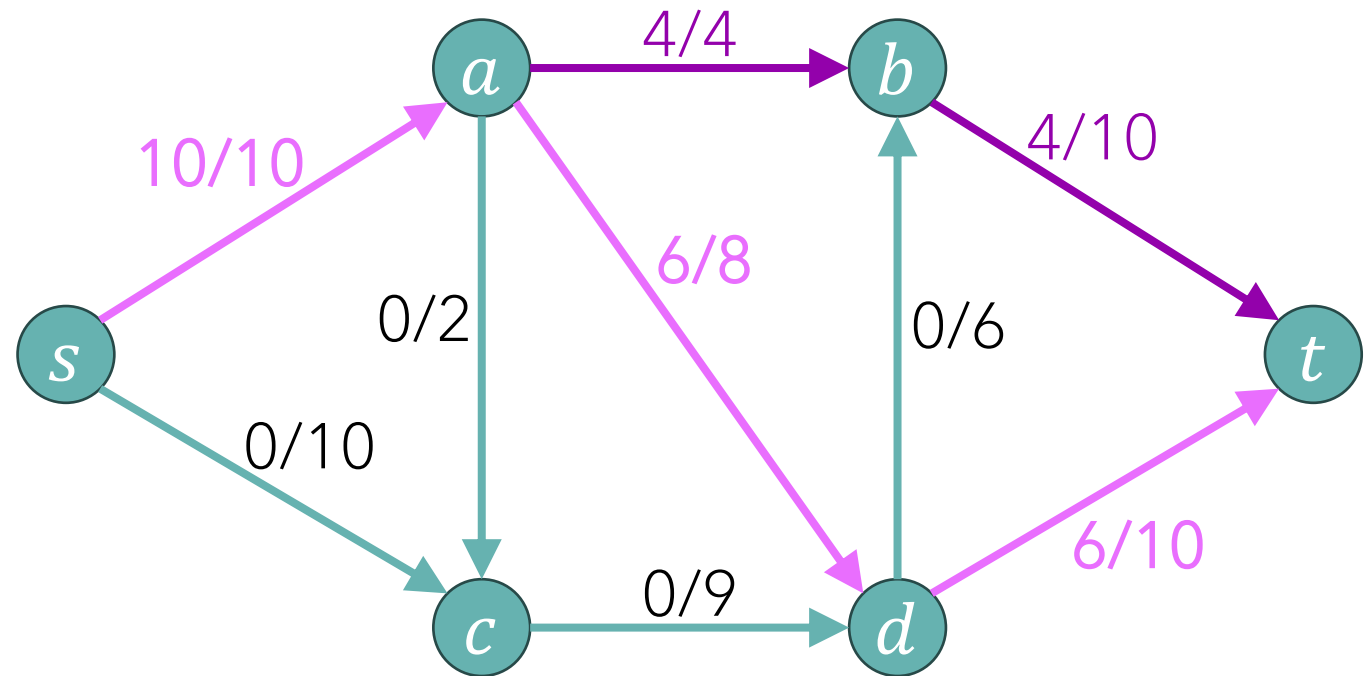
Step 1: $s \rightarrow a \rightarrow b \rightarrow t$ for 4



EDMONDS-KARP

Step 1: $s \rightarrow a \rightarrow b \rightarrow t$ for 4

Step 2: $s \rightarrow a \rightarrow d \rightarrow t$ for 6

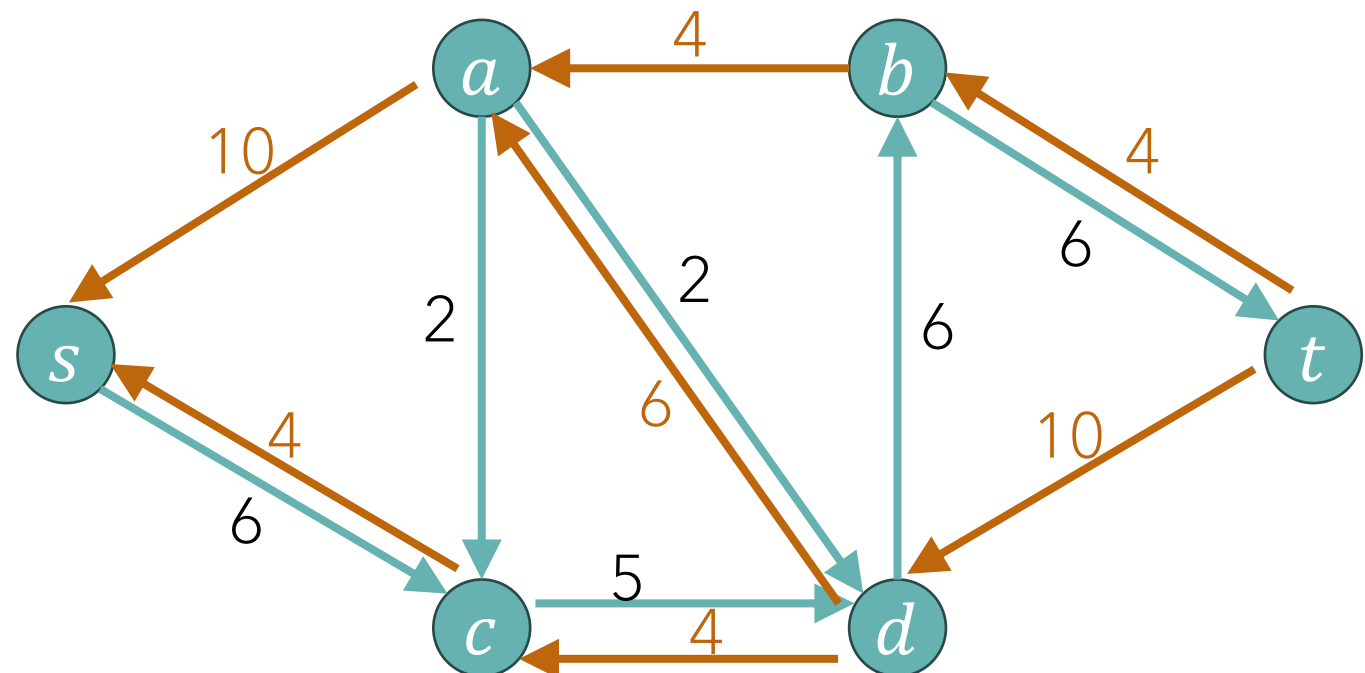
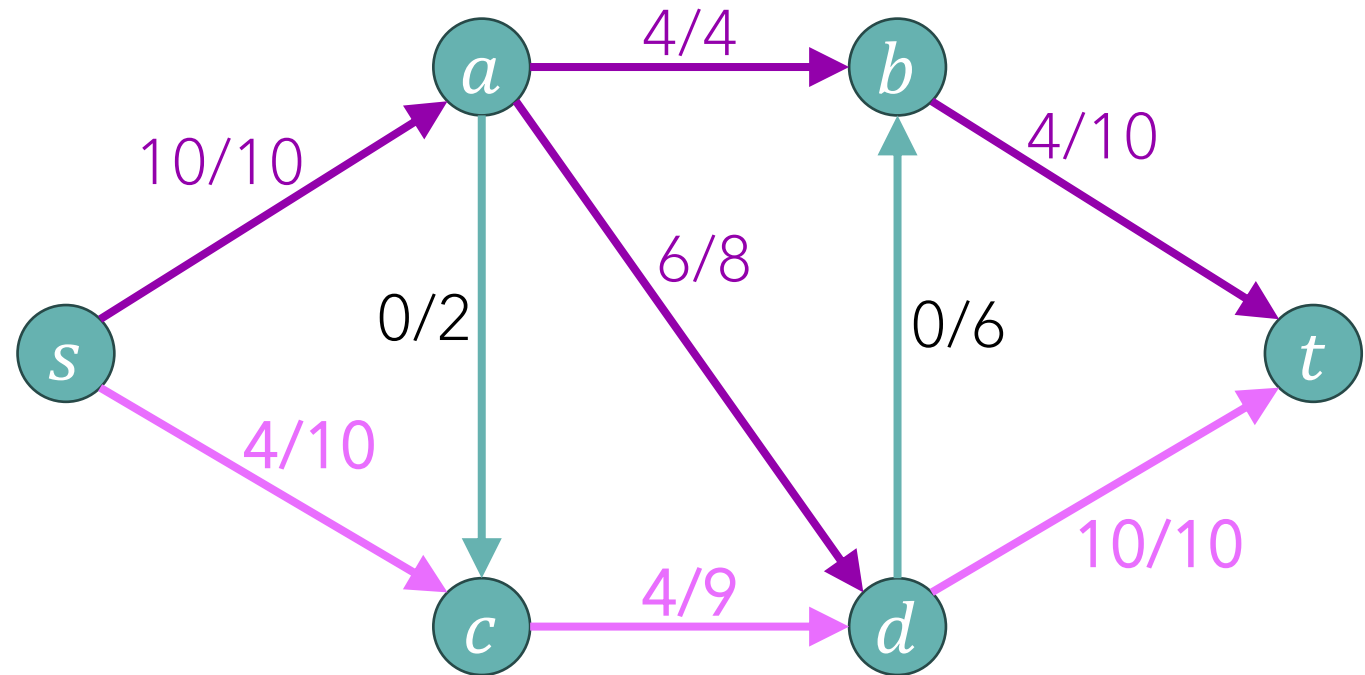


EDMONDS-KARP

Step 1: $s \rightarrow a \rightarrow b \rightarrow t$ for 4

Step 2: $s \rightarrow a \rightarrow d \rightarrow t$ for 6

Step 3: $s \rightarrow c \rightarrow d \rightarrow t$ for 4



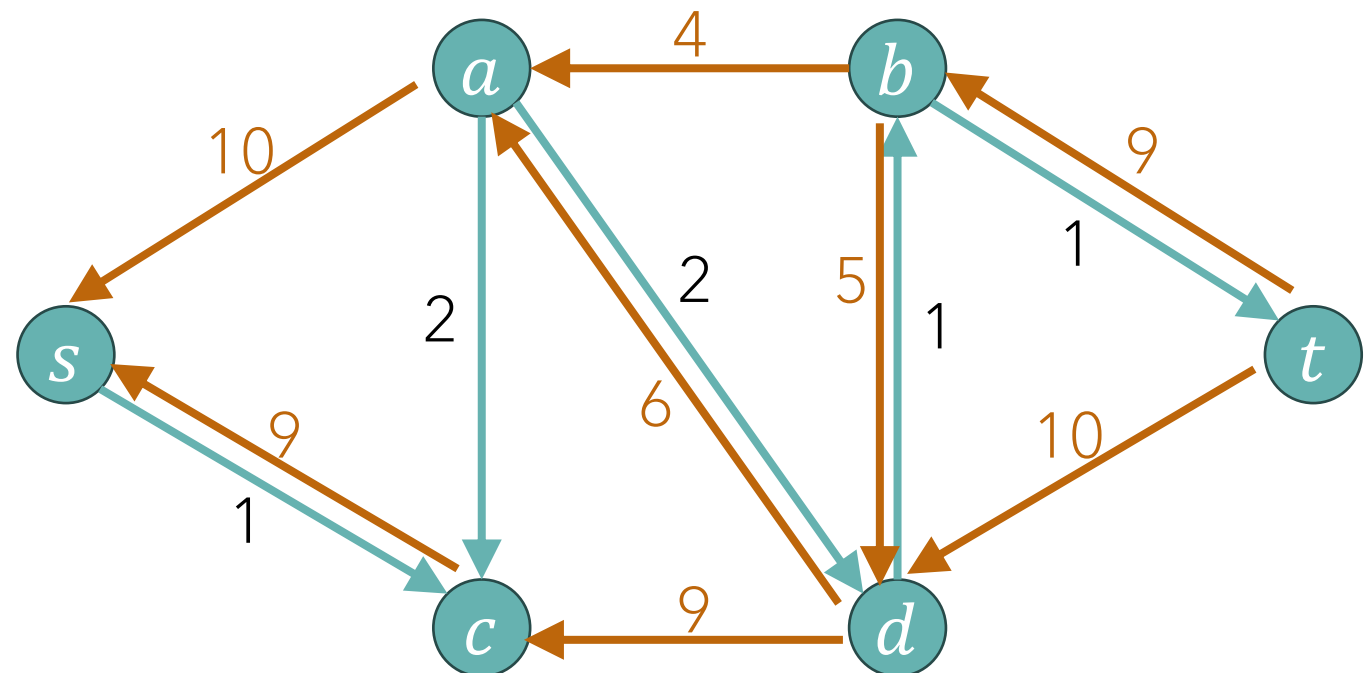
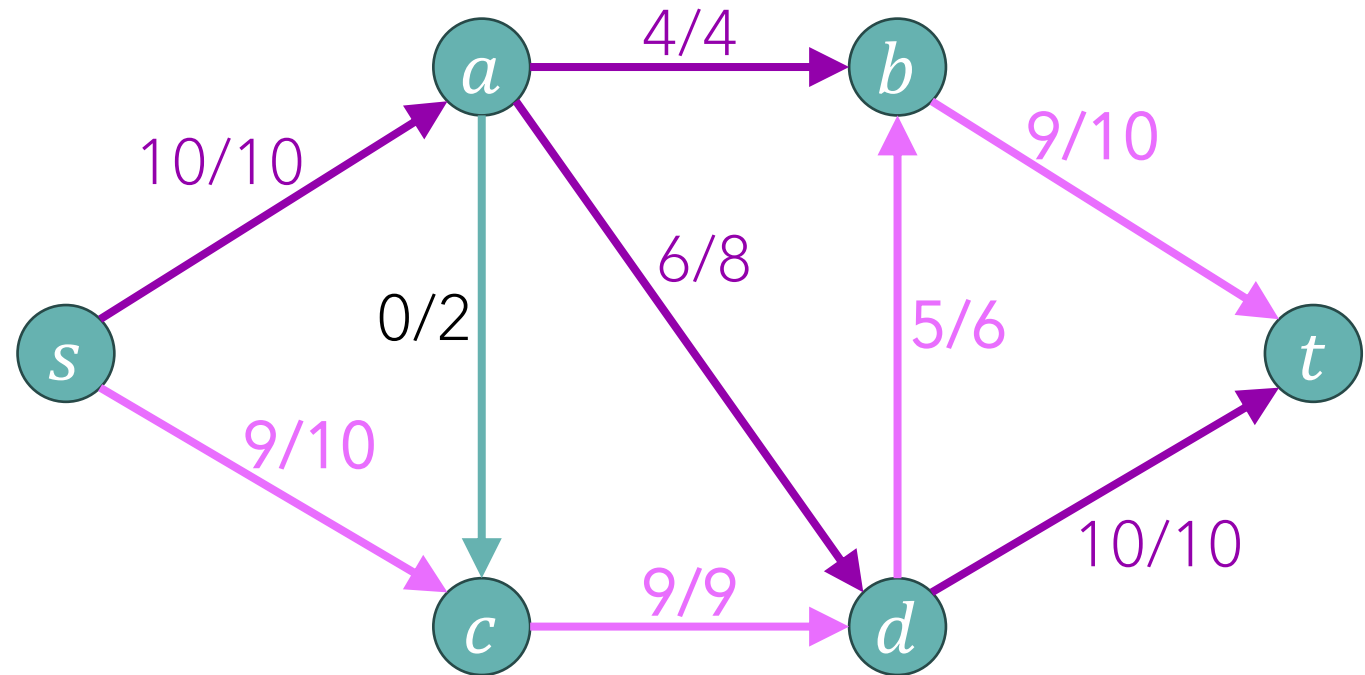
EDMONDS-KARP

Step 1: $s \rightarrow a \rightarrow b \rightarrow t$ for 4

Step 2: $s \rightarrow a \rightarrow d \rightarrow t$ for 6

Step 3: $s \rightarrow c \rightarrow d \rightarrow t$ for 4

Step 4: $s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$ for 5



EDMONDS-KARP

Step 1: $s \rightarrow a \rightarrow b \rightarrow t$ for 4

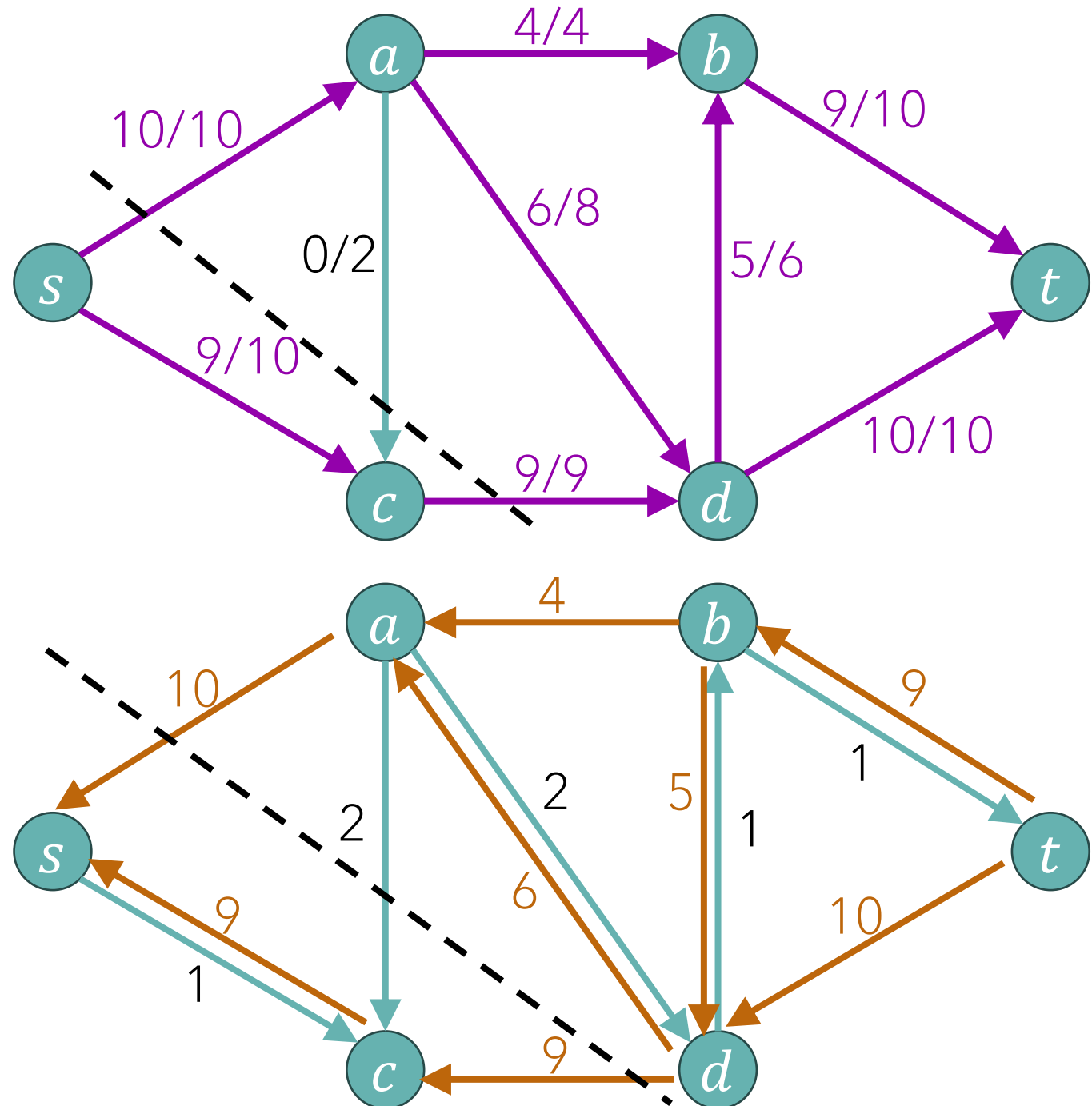
Step 2: $s \rightarrow a \rightarrow d \rightarrow t$ for 6

Step 3: $s \rightarrow c \rightarrow d \rightarrow t$ for 4

Step 4: $s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$ for 5

And we're done. There are no more augmenting paths in the residual graph.

In the final residual, c is the only node reachable from s , so that's the cut.



BEFORE YOU LEAVE

Please complete this feedback form about today's class.

